

Red Hat build of Keycloak Administration and Security

Version 1.0.0, 2026-07-02

Home

Introduction

Course Title: Red Hat build of Keycloak Administration and Security

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Introduce Red Hat build of Keycloak
- Install and Configure Red Hat build of Keycloak
- Authentication and Authorization
- Identity Brokering and User Federation
- Red Hat build of Keycloak on Red Hat OpenShift
- Extending and Templating Keycloak
- Comprehensive Review

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Introduce Red Hat build of Keycloak

Introduce Red Hat build of Keycloak

Red Hat build of Keycloak is an enterprise-ready identity and access management solution designed to secure modern applications and services. It simplifies security by centralizing authentication and authorization, allowing developers to offload identity management from their applications.

Key features and concepts include:

- **Support for Industry Standards:** Implements widely adopted protocols including OpenID Connect (OIDC), OAuth 2.0, and SAML 2.0 for seamless integration.
- **Centralized Security:** Provides a unified platform for managing user authentication, role-based access control (RBAC), and permission-based authorization.
- **Security Mechanisms:** Utilizes robust token-based authentication to ensure secure communication and credential isolation.
- **Architectural Flexibility:** Designed to support diverse operational environments, ranging from local development setups to scalable, containerized deployments on Red Hat OpenShift.

Overview of Red Hat build of Keycloak features

Overview of Red Hat build of Keycloak features

Red Hat build of Keycloak is an open-source Identity and Access Management (IAM) solution designed for modern applications and services. It provides a centralized security layer that allows developers and administrators to secure their applications with minimal effort, offloading the complexity of authentication and authorization to a dedicated, high-performance platform.

Core Features

Red Hat build of Keycloak simplifies security through several integrated feature sets:

- **Single Sign-On (SSO):** Users authenticate once to access multiple independent applications. This reduces password fatigue and centralizes session management.
- **Identity Brokering:** Acts as a bridge between your applications and external Identity Providers (IdPs). It allows users to authenticate using existing accounts from third-party services like GitHub, Google, or enterprise-grade OIDC/SAML providers.
- **User Federation:** Enables seamless integration with existing user directories, such as LDAP or Active Directory, without requiring a migration of user credentials to the Keycloak database.
- **Standardized Protocols:** Full support for industry-standard identity protocols ensures interoperability across diverse technology stacks.
- **Centralized Management:** A robust Admin Console and Admin CLI provide a unified interface to manage users, roles, permissions, and security policies.
- **Customization and Extensibility:** Offers flexible templating for user-facing interfaces (Login,

Registration) and an SPI (Service Provider Interface) for extending server functionality.

Supported Security Standards

Red Hat build of Keycloak is built on a foundation of open standards, ensuring compatibility with modern web and mobile architectures. The platform implements the following key specifications:

Standard	Description
OpenID Connect (OIDC)	An identity layer on top of the OAuth 2.0 protocol, allowing clients to verify the identity of the end-user.
OAuth 2.0	An authorization framework that enables third-party applications to obtain limited access to an HTTP service.
SAML 2.0	An XML-based standard for exchanging authentication and authorization data between an identity provider and a service provider.

Operational Concepts

Red Hat build of Keycloak utilizes specific mechanisms to ensure secure identity management:

- **Credential Isolation:** The system maintains a strict separation between the application and the user's credentials. Applications never touch raw user passwords; instead, they receive security tokens (typically JSON Web Tokens or JWTs) after successful authentication.
- **Token-based Security:** Once a user is authenticated, Keycloak issues tokens that the application can use to verify user identity and scope of access (claims), reducing the need for constant back-and-forth communication with the identity server.
- **Real-time Session Management:** Administrators can monitor and revoke user sessions globally across all connected applications directly from the Admin Console.

Knowledge Check

1. *What is the primary benefit of using Identity Brokering in Red Hat build of Keycloak?*

Identity Brokering allows your applications to delegate authentication to external providers (like social media or enterprise IdPs), enabling users to log in using their existing external accounts without requiring custom integration code for each provider.

2. *Which protocol is an identity layer built on top of OAuth 2.0?*

OpenID Connect (OIDC).

Single Sign-On standards (OIDC, OAuth 2.0, SAML 2.0)

Single Sign-On Standards (OIDC, OAuth 2.0, SAML 2.0)

Single Sign-On (SSO) enables users to authenticate once and gain access to multiple independent applications. Red Hat build of Keycloak (RHBK) acts as the Identity and Access Management (IAM) system that orchestrates this trust relationship using standardized protocols.

Understanding SSO Protocols

SSO relies on established protocols for the interaction between the user agent (typically a web browser), the secured applications, and the IAM system.

OpenID Connect (OIDC) and OAuth 2.0

While often discussed together, it is important to distinguish their roles:

- **OAuth 2.0:** Primarily an **authorization** framework. It allows a third-party application to obtain limited access to an HTTP service, either on behalf of a resource owner or by allowing the third-party application to obtain access on its own behalf.
- **OpenID Connect (OIDC):** An **authentication** layer built on top of the OAuth 2.0 framework. It allows clients to verify the identity of the end-user based on the authentication performed by an Authorization Server (like RHBK).

OIDC was finalized in 2014 and is designed for modern web architectures. It is the preferred choice for mobile applications, smart devices, and Single Page Applications (SPAs) due to its lightweight nature.

Security Assertion Markup Language (SAML) 2.0

SAML 2.0 is a mature, XML-based standard for exchanging authentication and authorization data.

- **Design:** Descended from SOAP and web service messaging, it is more verbose than OIDC.
- **Mechanism:** It relies on the exchange of XML documents, known as **SAML Assertions**, which are digitally signed and encrypted to ensure integrity and confidentiality.
- **Usage:** SAML is deeply embedded in enterprise environments. It is ideal for traditional web applications where XML processing is already a standard requirement.

Feature	OIDC	SAML 2.0
Data Format	JSON (JWT)	XML
Primary Use	AuthN + AuthZ	AuthN + AuthZ
Best for	Mobile, SPAs, Modern Web	Legacy Enterprise Apps
Maturity	Modern (2014)	Mature (2005)

Key Architectural Considerations

When implementing SSO with RHBK, keep the following in mind:

- **Protocol Coexistence:** RHBK is highly flexible; it can support both OIDC and SAML within the same realm, allowing you to bridge modern mobile applications (OIDC) and legacy enterprise software (SAML) simultaneously.
- **Credential Isolation:** By offloading authentication to the RHBK server, the individual applications never handle the user's primary credentials. Instead, they receive tokens (in OIDC) or assertions (in SAML) that verify the user's identity and provide necessary claims (like role mappings).

- **Communication Flow:** The SSO flow typically follows a "Redirect" pattern where the browser is sent to the IAM system, the user authenticates, and is then redirected back to the application with a proof of identity.

Summary Quiz: SSO Standards

1. *Which protocol is better suited for Single Page Applications (SPAs) and mobile devices?*

OIDC (OpenID Connect) is designed specifically for these architectures, whereas SAML presents challenges for non-browser-based or SPA environments.

2. *What is the primary difference between OAuth 2.0 and OIDC?*

OAuth 2.0 is an authorization framework, while OIDC is an identity (authentication) layer built on top of OAuth 2.0.

3. *Why does SAML use XML?*

SAML is derived from older web service standards (SOAP/XML) and uses XML documents to exchange assertions between the identity provider and the service provider.

Architecture and operational concepts

Architecture and Operational Concepts

The Red Hat build of Keycloak is designed as a high-performance, identity and access management (IAM) solution. Its architecture is built to support modern applications using standard protocols, ensuring that authentication and authorization are decoupled from the application logic.

Core Architectural Components

Red Hat build of Keycloak operates based on a few fundamental architectural pillars:

- **Realms:** The primary unit of isolation. A realm acts as a "space" where you manage a set of users, credentials, roles, and groups. An application is typically mapped to one realm.
- **Clients:** These are the entities (applications or services) that request Keycloak to authenticate a user. Clients can be web applications, mobile apps, or backend services.
- **Identity Providers (IdPs):** Keycloak can act as an identity provider itself, or it can delegate authentication to third-party providers (like Google, GitHub, or an external OIDC/SAML server) through Identity Brokering.
- **User Federation:** Instead of storing all users in the Keycloak database, you can connect to external user stores like LDAP or Active Directory, allowing Keycloak to pull user data on demand.

Operational Concepts: Organizations and Multi-tenancy

A key operational shift in modern Red Hat build of Keycloak is the concept of **Organizations**.

Organizations address complex Business-to-Business (B2B) use cases by enabling multi-tenancy within a single realm. While a realm provides global isolation, organizations provide a scoped, hierarchical structure for managing members from specific partners or customers.

Hierarchical Group Management

Within an organization, you can define groups to mirror your internal structure (e.g., `/Engineering/Backend` or `/Sales/NorthAmerica`). * **Isolation:** Organization groups are exclusive to their parent organization. * **Consistency:** Because they are scoped to the organization, you can have identical group paths in different organizations without conflict, as each possesses a unique internal identifier.

Security Token Mechanisms

Keycloak relies on standard tokens to communicate identity and authorization data:

- **Identity Tokens (ID Tokens):** Provide information about the authenticated user (OIDC).
- **Access Tokens:** Used by applications to access protected resources. These contain claims that describe the user's roles and permissions.
- **Refresh Tokens:** Allow clients to obtain new access tokens without forcing the user to re-authenticate.

By leveraging these tokens, Red Hat build of Keycloak ensures **credential isolation**. The application never handles the user's password; it only receives a digitally signed token that verifies the user's identity and their organizational context.

Summary of Operational Workflow

1. **Request:** A user attempts to access an application.
2. **Redirect:** The application redirects the user to the Keycloak instance.
3. **Authentication:** Keycloak authenticates the user (via local credentials, federated identity, or organizational brokering).
4. **Token Issuance:** Keycloak issues an ID and Access Token, which may include organization-specific claims.
5. **Authorization:** The application uses these tokens to enforce fine-grained access control based on the user's roles and organizational membership.



When integrating third-party partners, treat the **Organization** as the primary boundary for managing the member lifecycle, enabling a unified security experience across your entire business ecosystem.

Security token mechanisms and credential isolation

Security token mechanisms and credential isolation

In distributed systems, ensuring that sensitive credentials remain isolated while identity and authorization information flow across trust domains is a foundational requirement. Red Hat build of Keycloak leverages robust token mechanisms to provide secure, scalable authentication and authorization.

Security Token Mechanisms

Red Hat build of Keycloak relies primarily on JSON Web Tokens (JWT) to communicate assertions between parties. These tokens are designed to be compact, URL-safe, and self-contained.

Token Types

- **Access Tokens:** These represent the authorization granted to the client. They contain scopes and user attributes necessary for resource servers to make access control decisions.
- **Identity Tokens (ID Tokens):** These provide information about the authentication event, such as who the user is and when they authenticated, typically following the OpenID Connect (OIDC) specification.
- **Refresh Tokens:** Used by clients to obtain new access tokens without requiring the user to re-authenticate, enabling longer-lived sessions without compromising security.

Proof of Possession (PoP)

To prevent "bearer token" abuse—where a stolen token can be used by any party—Red Hat build of Keycloak supports **Proof of Possession**.

When a client uses a token to access a protected resource, it must cryptographically sign the request using its private key. The Resource Server verifies this signature against the fingerprint embedded in the access token. This ensures that even if a token is intercepted, it is useless to an attacker who does not possess the corresponding private key.

Credential Isolation and Identity Chaining

When applications span multiple trust domains, each with its own authorization server, we must avoid sharing raw user credentials (like passwords) across boundaries. Instead, we use **Identity and Authorization Chaining**.

This approach follows these principles: 1. **Token Exchange (RFC8693):** Allows a client to exchange a token from one domain for a new token suitable for a different domain. 2. **JWT Profile (RFC7523):** Provides a mechanism for client authentication using JWTs instead of shared secrets.

The Chaining Workflow

In a scenario where a client in **Domain A** needs to access a resource in **Domain B**:

- a. The client in **Domain A** exchanges its existing token with its local authorization server for a JWT authorization grant.
- b. The local authorization server validates the request and issues the JWT grant.
- c. The client presents this JWT grant to the authorization server in **Domain B**.
- d. The authorization server in **Domain B** validates the grant and issues a specific access token for **Domain B**.
- e. The client uses this new access token to access the protected resource in **Domain B**.

This process isolates the user's primary credentials within **Domain A**. **Domain B** only ever sees an

authorization grant that confirms the identity or authority of the client, never the user's original authentication credentials.

Hands-on Activity: Configuring Token Lifespans

To maintain a strong security posture, you must minimize the time-to-live (TTL) for tokens, ensuring that if a token is compromised, its window of utility is limited.

1. **Step 1:** Log in to the Red Hat build of Keycloak Admin Console.
2. **Step 2:** Select your Realm from the dropdown menu.
3. **Step 3:** Navigate to **Realm settings > Tokens**.
4. **Step 4:** Configure the following:
 - **Access Token Lifespan:** Set to **5 minutes** (default is usually longer; shortening this increases security).
 - **SSO Session Idle:** Set to **30 minutes**.
5. **Step 5:** Click **Save**.



Shortening token lifespans forces more frequent validation against the authorization server, but it must be balanced against the performance impact on your network and the overhead on the Keycloak server.

Knowledge Check

- **Why is Proof of Possession (PoP) more secure than a standard bearer token?**
- **How does token exchange prevent the need to share user passwords across multiple trust domains?**

Quiz: Describe Keycloak Systems

Quiz: Describe Keycloak Systems

This section provides a short quiz to test your understanding of why identity and access management (IAM) systems like Red Hat build of Keycloak are essential in modern enterprise architectures.

1. Which of the following best describes the primary challenge that Red Hat build of Keycloak solves in a microservices architecture?
 - a) Managing physical server hardware and network cabling.
 - b) Centralizing authentication and authorization to prevent "siloe" user stores across multiple applications.
 - c) Providing a database for storing unstructured raw binary files.
 - d) Writing the backend business logic for third-party financial applications.

Correct Answer: b Explanation: Red Hat build of Keycloak acts as a centralized Identity Provider (IdP), allowing developers to offload authentication and authorization logic, ensuring that users have a unified identity across disparate services.

1. In the context of "Credential Isolation," why is it a security best practice to use an IAM system like Keycloak rather than storing passwords directly within each individual application?
 - a) It makes it easier for developers to manually reset passwords via database queries.
 - b) It reduces the attack surface by ensuring that application code never needs to handle or store sensitive user credentials directly.
 - c) It increases the speed of the application's UI rendering.
 - d) It eliminates the need for any form of encryption.

Correct Answer: b Explanation: By centralizing credentials in a hardened, dedicated security product, applications only handle security tokens (like OIDC ID tokens) rather than raw credentials, significantly reducing the risk of credential theft during a breach of a single application.

1. Which of the following protocols is natively supported by Red Hat build of Keycloak to enable Single Sign-On (SSO) across applications? (Select all that apply)
 - a) OIDC (OpenID Connect)
 - b) OAuth 2.0
 - c) SAML 2.0
 - d) FTP (File Transfer Protocol)

Correct Answer: a, b, and c Explanation: Keycloak is designed to support industry-standard identity protocols, including OIDC, OAuth 2.0, and SAML 2.0, allowing for seamless interoperability between various platforms and services.

Quiz: Introduce Red Hat build of Keycloak

Quiz: Introduce Red Hat build of Keycloak

This quiz tests your foundational knowledge of the Red Hat build of Keycloak architecture, its purpose, and the identity standards it supports.

Question 1: Which of the following identity and access management standards are natively supported by Red Hat build of Keycloak?

- ☐ A) OIDC (OpenID Connect)
- ☐ B) OAuth 2.0
- ☐ C) SAML 2.0
- ☐ D) All of the above

Question 2: What is the primary purpose of using Red Hat build of Keycloak in an enterprise environment?

- ☐ A) To manage database migrations for containerized applications.
- ☐ B) To provide centralized identity management, single sign-on (SSO), and secure access to applications and services.
- ☐ C) To act as a direct replacement for a web server like Nginx or Apache.
- ☐ D) To perform automated vulnerability scanning on the host operating system.

Question 3: In the context of Red Hat build of Keycloak, what does "credential isolation" imply?

- ☐ A) Storing all user passwords in a plain-text configuration file for easy administration.
- ☐ B) Ensuring that client applications do not handle user credentials directly, reducing the attack surface.
- ☐ C) Forcing every user to use the same shared credentials for every service.
- ☐ D) Isolating the Keycloak database from the network to prevent all external logins.

Question 4: Which of the following is a core operational concept of Red Hat build of Keycloak?

- ☐ A) It relies on client applications to store user identities locally.
- ☐ B) It acts as an Identity Provider (IdP) that issues security tokens (like JWTs) to authenticated users.
- ☐ C) It is designed to be installed locally on every individual user's machine.
- ☐ D) It is a hardware-based security module that does not require network connectivity.

Answer Key

- **Question 1: D) All of the above.** Red Hat build of Keycloak is designed as a modern IAM solution that supports industry-standard protocols including OIDC, OAuth 2.0, and SAML 2.0 to ensure interoperability.
- **Question 2: B) To provide centralized identity management, single sign-on (SSO), and secure access.** By centralizing authentication, you reduce the need for users to remember multiple credentials and enable consistent security policies across services.
- **Question 3: B) Ensuring that client applications do not handle user credentials directly.** This is a fundamental security benefit of OIDC and OAuth 2.0, where Keycloak manages the password exchange, and applications receive tokens instead of raw user credentials.
- **Question 4: B) It acts as an Identity Provider (IdP) that issues security tokens (like JWTs) to authenticated users.** Keycloak authenticates the user and provides a token that the application can then trust to identify the user and their authorized permissions.

Install and Configure Red Hat build of Keycloak

Install and Configure Red Hat build of Keycloak

Installing and configuring the Red Hat build of Keycloak 26.6 involves setting up the identity and access management server to manage user authentication and authorization. This process ensures the environment is prepared for securing applications using industry-standard protocols.

Key installation and configuration tasks include:

- **Environment Setup:** Preparing the infrastructure to meet the requirements for development or production deployments.
- **Core Configuration:** Utilizing the Admin Console and Admin CLI to define realms, clients, and identity providers.
- **System Management:** Managing authentication flows and user federation sources to integrate with existing identity stores such as LDAP or Active Directory.
- **Operational Deployment:** Configuring the server for containerized environments, including deployment on Red Hat OpenShift.

Development mode installation requirements

Development Mode Installation Requirements

To get started with the Red Hat build of Keycloak, the development mode is designed for rapid prototyping and local testing. This mode prioritizes ease of use and developer productivity by relaxing several security and performance constraints required for production environments.

Development Mode Overview

Development mode is intended strictly for local development and initial exploration. It allows developers to quickly spin up an instance to test configurations, develop custom themes, or experiment with identity providers without needing to configure complex networking or high-availability infrastructure.

Default Configuration

When starting the Red Hat build of Keycloak in development mode, the system automatically applies the following defaults:

- **HTTP Protocol:** HTTP is enabled by default. Note that this is insecure for production use and should not be used for transferring sensitive credentials over public networks.
- **Hostname Resolution:** Strict hostname resolution is disabled. This simplifies local testing by allowing access via `localhost` or non-standard host configurations.
- **Cache Configuration:** The cache is set to `local`. There is no distributed cache mechanism

enabled, which means the instance cannot share state with other nodes in a high-availability cluster.

- **Theme and Template Caching:** Both theme-caching and template-caching are disabled. This is critical for theme development, as it ensures that changes made to HTML templates or CSS files are reflected immediately upon refreshing the browser, without requiring a server restart.

System Requirements and Considerations

Before installing in development mode, ensure your environment meets the following criteria:



While development mode is convenient, you must refer to the **Red Hat Knowledgebase Article on supported configurations** to review the official requirements regarding Java versions and compatible operating systems. Even for local development, using a supported OpenJDK version is required to ensure consistent behavior.

Directory Structure Understanding

Upon installation, familiarize yourself with the installation root directory. Understanding the file hierarchy is essential for managing your development instance:

- **bin/:** Contains the `kc.sh` (or `kc.bat`) scripts required to start and stop the server.
- **conf/:** Contains the `keycloak.conf` file. While development mode uses defaults, you can override specific settings here if needed.
- **providers/:** The location for deploying custom extensions or SPI (Service Provider Interface) implementations.
- **themes/:** The directory where you will create and modify custom themes for the login and account management pages.

Hands-on Exercise: Launching in Development Mode

Follow these steps to verify your installation in development mode.

1. Navigate to the installation directory:

```
cd /path/to/keycloak-installation
```

2. Start the server:

```
./bin/kc.sh start-dev
```

3. **Verify the logs:** Observe the console output. You will see messages indicating that the server is running in "dev" profile and that development-specific features (like disabled strict hostname resolution) are active.
4. **Access the Admin Console:** Open your browser and navigate to <http://localhost:8080>. You will

be prompted to create the initial administrative user if this is your first time launching the instance.



If you are developing themes, remember that because template-caching is disabled, you can edit files in the `themes/` directory and simply reload your browser to see changes instantly.

Admin Console exploration and configuration

Admin Console exploration and configuration

The Red Hat build of Keycloak Admin Console provides a centralized, web-based interface for managing all aspects of the identity server. It allows administrators to configure realms, manage users, define client applications, and establish authentication policies without needing to interact directly with the underlying configuration files or the command-line interface.

Overview of the Admin Console

The Admin Console is divided into several primary navigation areas that represent the hierarchical structure of a Keycloak deployment:

- **Realm Selector:** Located in the top left, this dropdown allows you to switch between different security realms. A realm is a space where you manage a set of users, credentials, roles, and groups.
- **Main Navigation Sidebar:** Contains the core management modules:
- **Manage:** Handles day-to-day operations like Users, Groups, Roles, Clients, and Identity Providers.
- **Configure:** Defines global settings, such as Realm Settings, Authentication Flows, Events, and User Federation.
- **Dashboard:** Provides a high-level view of recent events and system health once you are logged in.

Accessing the Admin Console

To begin exploration, you must sign in to the Admin Console using the initial administrator credentials established during the installation process.

Guided Exercise: Explore and Configure the Admin Console

1. Navigate to the URL provided by your administrator (typically <http://localhost:8080/admin>).
2. Enter the initial administrator credentials (username and password) created during the initial setup.
3. Upon successful authentication, the **Master** realm dashboard displays.
4. Click on **Realm Settings** in the left-hand menu.
5. Explore the tabs (General, Login, Email, Themes, etc.) to see the configurable options available for the current realm.

Configuration Concepts

When configuring the Admin Console, it is important to understand the scope of your changes:

- **Realm-level configuration:** Most settings are scoped to a specific realm. For example, changing a theme or an authentication flow in the "Development" realm will not affect the "Production" realm.
- **Credential Isolation:** Administrators should always operate using the Principle of Least Privilege. Use the **Admin Console Access Control** features to ensure that administrative users have only the permissions necessary for their specific roles.

Hands-on Activity: Configuring Basic Realm Settings

In this activity, you will perform a basic configuration task to familiarize yourself with the interface.

Steps to update Realm display name:

1. In the Admin Console, ensure the 'Master' realm is selected.
2. Select 'Realm Settings' from the left navigation menu.
3. Under the 'General' tab, locate the 'Display name' field.
4. Enter a descriptive name (e.g., "Corporate Identity Provider").
5. Click 'Save'.



If you encounter issues accessing specific configuration sections, verify that your account has the appropriate `realm-admin` or `master-admin` roles assigned under the **Users > Role Mappings** section of the Admin Console.

Next Steps

Now that you have navigated the interface and performed a basic configuration, the next modules will cover using the Admin CLI to perform these same operations programmatically and managing identity brokering configurations.

Admin CLI operations

Admin CLI operations

The Red Hat build of Keycloak (RHBK) includes a powerful command-line utility, `kcadm.sh`, designed to automate administrative tasks. This tool acts as a wrapper around the Admin REST API, allowing administrators to perform CRUD (Create, Read, Update, Delete) operations without needing to navigate the web-based Admin Console.

Understanding the Admin CLI Architecture

The Admin CLI is a Java-based utility that maps command-line arguments directly to the underlying REST API endpoints of the Keycloak server. By using `kcadm.sh`, you can script environment setups, automate user provisioning, and manage configurations in a reproducible manner.

Usage Syntax

The general structure for executing commands via the CLI is:

```
[rhssso@sso ~]$ kcadm.sh OPERATION ENDPOINT [ ARGUMENTS ]
```

- **OPERATION:** The action to perform (**create**, **get**, **update**, **delete**, etc.).
- **ENDPOINT:** The target resource path in the Keycloak Admin REST API (e.g., **realms**, **users**, **groups**).
- **ARGUMENTS:** Specific parameters, such as JSON payloads or configuration flags.

Core Admin CLI Operations

The utility supports several essential operations to facilitate lifecycle management of your identity environment:

- **CRUD Operations:** These map directly to HTTP verbs:
 - **create:** Maps to **POST** (e.g., creating a new realm or user).
 - **get:** Maps to **GET** (e.g., retrieving configuration data).
 - **update:** Maps to **PUT** (e.g., modifying existing resource attributes).
 - **delete:** Maps to **DELETE** (e.g., removing a user or client).
- **config:** Used to set up authentication credentials, target server URLs, and other environment-specific settings.
- **Role Management:** Specialized commands like **get-roles**, **add-roles**, and **remove-roles** simplify the mapping of permissions to users or groups.
- **set-password:** A convenient shorthand to update user credentials without constructing a full JSON update request.

Guided Exercise: Exploring Admin CLI Operations

In this exercise, we will look at how to initialize the CLI environment and retrieve server information.

Step 1: Configure the CLI session

Before interacting with the server, you must authenticate your CLI session.

```
# Configure the server connection and authenticate
[rhssso@sso ~]$ kcadm.sh config credentials --server http://localhost:8080/ --realm
master --user admin --password <your-password>
```

Step 2: Retrieve Realm Information

Use the **get** operation to list details about your current environment.

```
# Get a list of all realms
[rhssso@sso ~]$ kcadm.sh get realms
```

Step 3: Cleanup

Once you have completed your exploration of the CLI, ensure that you conclude the lab properly to maintain environment stability.

```
[student@workstation ~]$ lab finish install-adminconsole
```

Troubleshooting and Best Practices

- **Help is Available:** If you are unsure of the arguments for a specific endpoint, use the `help` command: `kcadm.sh help`.
- **JSON Payloads:** For `create` or `update` operations, you can pass a JSON file using the `-f` flag to manage complex configurations easily: `kcadm.sh create users -f user-data.json`.
- **Credential Security:** Avoid running commands containing passwords in your shell history. Use environment variables or configuration files where possible in production environments.

Guided Exercise: Red Hat build of Keycloak Installation

Guided Exercise: Red Hat build of Keycloak Installation

In this exercise, you will perform a development-mode installation of the Red Hat build of Keycloak. This environment is ideal for initial exploration, testing, and learning the core administrative concepts.

Prerequisites

- A system with Java 17 or later installed.
- Access to the Red Hat build of Keycloak distribution (ZIP or TAR file).
- Network access to allow binding to the local loopback address.

Step 1: Extracting the Distribution

Before running the server, you must extract the downloaded distribution to a preferred directory.

1. Open your terminal.
2. Navigate to your desired installation directory (e.g., `/opt/keycloak` or your local home directory).
3. Extract the archive:

```
# For .tar.gz files
```

```
tar -xzf keycloak-22.0.x.tar.gz
```

```
# For .zip files  
unzip keycloak-22.0.x.zip
```

Step 2: Starting in Development Mode

Red Hat build of Keycloak provides a "development" mode that is pre-configured with default settings, including an embedded H2 database and self-signed certificates.

1. Navigate into the extracted **bin** directory.
2. Start the server using the **start-dev** command:

```
./kc.sh start-dev
```



The **start-dev** flag is strictly for development and testing. It automatically sets the hostname to localhost and enables HTTP, which is insecure for production environments.

Step 3: Verifying the Installation

Once the server logs indicate that the service has started successfully, verify that the instance is reachable.

1. Open your web browser.
2. Navigate to: <http://localhost:8080>
3. You should see the **Welcome to Keycloak** landing page.

Step 4: Creating the Initial Admin User

The server does not ship with a pre-created administrator account for security reasons. You must create the initial account through the Admin Console or the command line.

1. On the landing page, click the **Administration Console** link.
2. Because no users exist, the console will prompt you to create an initial admin account.
3. Fill in the required fields:
 - **Username:** **admin**
 - **Password:** **your-secure-password**
 - **Password Confirmation:** **your-secure-password**
4. Click **Create**.
5. Once created, click the **Back to Administration Console** link to log in with your new credentials.

Troubleshooting

If the server fails to start, check the following: * **Port Conflict:** By default, Keycloak binds to port **8080**. Ensure no other services (like Apache or Tomcat) are using this port. You can change the port using the `--http-port` flag. * **Java Version:** Ensure `java -version` returns version 17 or higher. * **Logs:** Check the logs directory (`/data/log`) if the startup process terminates unexpectedly.

Summary

You have successfully deployed a development instance of the Red Hat build of Keycloak. In the next section, we will explore the Admin Console and perform initial configuration tasks.

Guided Exercise: Explore and Configure the Admin Console

Guided Exercise: Explore and Configure the Admin Console

The Red Hat build of Keycloak Admin Console is the central web-based interface for managing your identity and access management environment. This exercise guides you through the navigation and core configuration areas of the console.

Objectives

- Access the Admin Console interface.
- Navigate the primary navigation menu.
- Identify the scope of Realms.
- Configure basic settings for a realm.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- Credentials for the initial administrator account created during installation.

Procedure

1. Accessing the Admin Console

1. Open your web browser and navigate to the Keycloak URL (typically <http://localhost:8080>).
2. Click the **Administration Console** link.
3. Authenticate using your administrator username and password.

2. Navigating the Admin Console

Once logged in, familiarize yourself with the layout:

- **Realm Selector (Top Left):** Keycloak is multi-tenant. This dropdown allows you to switch between different security contexts (Realms). By default, you will see the **Master** realm.

- **Primary Navigation (Left Sidebar):** This menu provides access to core management functions, including:
- **Configure:** Manage Realm settings, Clients, Identity Providers, and User Federation.
- **Manage:** Oversee Users, Groups, Roles, and Sessions.
- **Events:** View login and admin audit logs.

3. Exploring Realm Settings

The **Master** realm is intended for administrative purposes. To configure your specific application environment, it is best practice to create a new realm.

1. Hover over the **Realm Selector** at the top left.
2. Click **Create realm**.
3. Enter a name for your realm (e.g., **CorporateApp**) and click **Create**.
4. In the left-hand menu, navigate to **Configure > Realm settings**.
5. Observe the tabs available:
 - **General:** Modify display names and login settings.
 - **Login:** Enable/disable features like "Remember Me," "User registration," and "Forgot password."
 - **Email:** Configure SMTP server settings to enable system-generated emails.
 - **Themes:** Customize the look and feel of your login and account pages.

4. Configure Session and Token Settings

1. While in **Realm settings**, click the **Tokens** tab.
2. Review the default settings for:
 - **SSO Session Idle:** The time a session can remain inactive.
 - **SSO Session Max:** The maximum duration of a session.
 - **Access Token Lifespan:** The duration of the JWT before it must be refreshed.
3. Make a minor adjustment to the **Access Token Lifespan** (e.g., change it to 5 minutes) and click **Save**.

Troubleshooting

- **Cannot see Realm Settings:** Ensure you are logged in with an account that has **manage-realm** permissions.
- **Changes not taking effect:** Always ensure you have clicked the **Save** button at the bottom of the form after making configuration changes.

Additional Resources

- For deep dives into granular permissions, refer to the **Admin Console Access Control** chapter in the official Red Hat build of Keycloak documentation.

Guided Exercise: Red Hat build of Keycloak Admin CLI

Guided Exercise: Red Hat build of Keycloak Admin CLI

The Red Hat build of Keycloak Admin CLI (`kcadm.sh`) is a powerful utility that allows administrators to perform administrative tasks, such as managing realms, clients, and users, directly from the terminal. This tool is essential for automation and scripting administrative operations.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- The `kcadm.sh` script (located in the `bin/` directory of your Keycloak installation).
- A user account with the `admin` role.

Step 1: Locating the Admin CLI

The `kcadm.sh` script is bundled with your Keycloak distribution. Navigate to your installation directory to verify the tool is available.

```
cd /path/to/keycloak/bin
ls kcadm.sh
```

Step 2: Authenticating with the Admin CLI

Before executing commands, you must authenticate as an administrator to establish a session. You will need the server URL, the realm, and your admin credentials.

1. Execute the authentication command:

```
./kcadm.sh config credentials --server http://localhost:8080/ --realm master --user
admin --password <your_admin_password>
```

2. Verify the session was successfully created:

```
./kcadm.sh get serverinfo
```

If successful, you will see a JSON output detailing the Keycloak server environment.

Step 3: Managing Resources

The `kcadm.sh` tool uses standard HTTP verbs (GET, POST, PUT, DELETE) to interact with the Keycloak REST API.

Listing Realms

To list all realms currently configured in your Keycloak instance:

```
./kcadm.sh get realms
```

Creating a New Realm

To create a new realm, you can pass a JSON configuration file. Create a file named `myrealm.json`:

```
{
  "realm": "my-new-realm",
  "enabled": true
}
```

Then, execute the following command:

```
./kcadm.sh create realms -f myrealm.json
```

Step 4: Troubleshooting

If you encounter errors, ensure the following:

- **URL Mismatch:** Ensure the `--server` URL matches the actual base URL of your Keycloak instance.
- **Permissions:** Verify that the user provided has `manage-realm` or higher permissions in the `master` realm.
- **Connectivity:** If Keycloak is running on OpenShift or a remote server, ensure your local environment has network access to the exposed API port.

Review Questions

1. What command is used to authenticate the `kcadm.sh` tool against a specific realm?
2. Which HTTP method does the `kcadm.sh` tool use by default when you provide a JSON file with the `create` command?
3. Where is the `kcadm.sh` script located within the standard Red Hat build of Keycloak installation?



For advanced configurations, including custom truststores or non-standard ports, refer to the **Red Hat build of Keycloak Admin CLI documentation** in the official product guide.

Lab: Install Red Hat build of Keycloak

Lab: Install Red Hat build of Keycloak

In this lab, you will perform a development-mode installation of the Red Hat build of Keycloak. This process prepares a local environment for administrative configuration and testing.

Prerequisites

- A supported Java Development Kit (JDK) installed (JDK 17 or later).
- The Red Hat build of Keycloak distribution archive.
- Approximately 1GB of available disk space.

Objectives

- Download and extract the Red Hat build of Keycloak server.
- Initialize the server in development mode.
- Access the Admin Console for initial validation.

Step-by-Step Instructions

Step 1: Extract the Distribution

Download the distribution archive from the Red Hat Customer Portal. Extract the contents to a local directory of your choice.

```
# Replace keycloak-version with your specific downloaded filename
tar -xvf rhbk-keycloak-version.tar.gz
cd rhbk-keycloak-version
```

Step 2: Start the Server in Development Mode

Development mode is designed for rapid testing and configuration. It automatically sets up a built-in H2 database and enables HTTP (without requiring TLS certificates for initial setup).

Run the start-dev command:

```
bin/kc.sh start-dev
```

Expected Output: The server will output startup logs. Look for the message **Keycloak ... started in (X) ms** and note the URL provided for the Admin Console, typically <http://localhost:8080>.

Step 3: Create the Initial Admin User

Because the server was started in **start-dev** mode, if no other users exist, you may need to define the administrative account via environment variables or follow the browser-based prompt if the system is fresh.

If prompted, navigate to <http://localhost:8080/admin> in your web browser.

Step 4: Validate the Installation

1. Open your browser and navigate to <http://localhost:8080>.
2. Click on the **Administration Console** link.

3. Log in using the credentials you defined.
4. Verify that you can view the default **master** realm dashboard.

Troubleshooting

- **Port Conflict:** If port **8080** is already in use, you can start the server on a different port using the **--http-port** flag:

```
bin/kc.sh start-dev --http-port 8081
```

- **Java Issues:** Ensure your **JAVA_HOME** environment variable is pointing to a valid JDK 17 or 21 installation by running **java -version**.

Review

- You have successfully installed the Red Hat build of Keycloak.
- You have verified the server is running in development mode.
- You have confirmed access to the Admin Console, which is the prerequisite for all subsequent configuration tasks.

Authentication and Authorization

Authentication and Authorization

Authentication and Authorization are the foundational pillars of the Red Hat build of Keycloak, ensuring that only verified users gain access to resources and hold the appropriate permissions.

- **User Authentication Flows:** Defines the mechanism and sequence of steps used to verify a user's identity, such as password-based login, multi-factor authentication, or identity brokering.
- **Authorization Models:** Implements Role-Based Access Control (RBAC) and Permission-Based Access Control to manage what authenticated users are allowed to perform within an application.
- **Session Customization:** Allows administrators to tailor authentication sessions, including timeout policies and adaptive security requirements, to meet specific organizational security standards.

User authentication flows

User Authentication Flows

In the Red Hat build of Keycloak (RHBK), an **Authentication Flow** serves as a programmable engine that dictates the sequence of actions, checks, and interactions a user must complete during critical identity events, such as logging in, registering a new account, or resetting a credential.

Rather than hard-coding security requirements, RHBK provides a modular framework where administrators can assemble "Execution" steps into a customized workflow.

Understanding Authentication Flows

An authentication flow is a container that organizes various tasks, which we refer to as **Executions**. These executions can be configured as **Required**, **Alternative**, **Disabled**, or **Conditional**.

Key Workflow Categories

- **Login Flow:** Defines the credential verification process (e.g., username/password, then OTP, then WebAuthn).
- **Registration Flow:** Governs the information gathering process for new users, including profile validation, terms and conditions acceptance, and bot-prevention mechanisms like reCAPTCHA.
- **Credential Reset Flow:** Manages the recovery process, ensuring users prove their identity before modifying security settings.

The Execution Logic

The power of RHBK flows lies in how executions are handled:

- **Required:** The user must successfully complete this step to proceed.

- **Alternative:** The user can choose from a list of options (e.g., choose either Google Login or Username/Password).
 - **Conditional:** The execution only runs if specific logic is met (e.g., only prompt for MFA if the user is connecting from an unknown IP address).
-

Guided Exercise: Exploring Authentication Flows

In this exercise, you will explore the default authentication flows provided by RHBK.

Prerequisites

- Red Hat build of Keycloak installed and running.
- Access to the Admin Console with realm management privileges.

Steps

1. Access the Flows Console:

- Log in to your Keycloak Admin Console.
- Select your target **Realm** from the dropdown menu.
- In the left-hand navigation menu, click **Authentication**.

2. Examine the Browser Flow:

- Locate the **browser** flow in the list. This is the default flow used for standard browser-based logins.
- Click the **Actions** menu (three dots) and select **Config** or simply click the flow name to expand the execution list.
- Observe the sequence of executions, such as "Cookie" (for existing sessions) and "Username Password Form."

3. Create a New Custom Flow:

- Click the **Create flow** button at the top right.
- Enter a name (e.g., **Custom-MFA-Flow**).
- Select **Basic flow** as the Flow Type.
- Click **Save**.

4. Add Executions:

- Within your new flow, click **Add execution**.
- Search for **OTP Form** and add it.
- Set the status to **Required** if you wish to enforce MFA for all users using this flow.

5. Assign the Flow:

- To use this new flow, go to the **Bindings** tab on the Authentication page.
- Change the **Browser flow** setting from the default **browser** flow to your new **Custom-MFA-Flow**.

Troubleshooting Insight

If you create a flow that prevents users from logging in (e.g., requiring an OTP that is not yet configured for a user), you can be locked out of the Admin Console. Always keep a separate administrative account or ensure your flow has a fallback mechanism (like "Alternative" options) before binding it to the Browser flow.

Role-based and permission-based authorization

Role-based and Permission-based Authorization

In the Red Hat build of Keycloak (RHBK), authorization is the critical process of determining whether a user, application, or service has permission to access specific resources. RHBK manages this by injecting access claims into OIDC tokens or SAML assertions, which applications then parse to make real-time access decisions.

Understanding Role-Based Access Control (RBAC)

RBAC is the primary strategy for managing access in RHBK. It centers on the concept of **roles**, which act as labels representing a category of users or applications authorized to perform specific tasks.

- **Roles as Categories:** Roles group users with shared requirements (e.g., **admin**, **manager**, **user**).
- **Decoupled Permissions:** Applications define the permissions associated with a role. By mapping users to these roles, administrators can grant consistent access levels across the organization without configuring individual permissions for every user.
- **Namespaces:** RHBK supports two types of role namespaces:
- **Realm Roles:** Global roles that apply to users across the entire realm.
- **Client Roles:** Roles scoped to a specific application (client), allowing for fine-grained control unique to that service.

User Role Mapping

A **user role mapping** creates the logical link between a user identity and a specific role.

- **Encapsulation:** When a user authenticates, RHBK includes their assigned roles within their security token (JWT/SAML).
- **Efficiency:** Because the authorization data travels with the token, the target application does not need to query the identity provider repeatedly, reducing latency and decoupling the application from the identity store.

Leveraging Groups for Scalability

While roles define **what** a user can do, **Groups** provide an organizational structure to manage **who** the users are. * Groups serve as collections of users. * You can assign roles and attributes to a group; any user added to that group automatically inherits those roles. This is a best practice for managing large-scale user populations where individual role assignment becomes administratively

burdensome.

Guided Exercise: Authorizing Users with RBAC

This exercise demonstrates how to create a role and map it to a user.

Step 1: Create a Realm Role

1. Log in to the **Admin Console**.
2. Select your realm from the top-left dropdown.
3. In the left-hand menu, click **Realm roles**.
4. Click **Create role**.
5. Enter **manager** as the Role name and click **Save**.

Step 2: Map the Role to a User

1. Click **Users** in the left-hand menu.
2. Select an existing user (or create a new one).
3. Select the **Role mapping** tab.
4. Click **Assign role**.
5. Filter by "Realm roles," select the **manager** role you created, and click **Assign**.

Step 3: Verification

- Observe the user's details. The user now possesses the **manager** role, which will be included in their next token issuance, allowing your applications to recognize them as an authorized manager.

Advanced Authorization Strategies

While RBAC is the standard, RHBK supports more sophisticated methods for complex environments:

- **Attribute-Based Access Control (ABAC):** Authorization decisions are made based on user or resource attributes (e.g., "Allow access if **user.department** equals ``finance``").
- **Fine-Grained Authorization:** RHBK allows you to define policies and permissions directly on resources, scopes, and objects, enabling you to secure specific APIs or UI elements based on runtime conditions.



When planning your authorization architecture, prioritize **RBAC** for general user-to-application access, and reserve **Fine-Grained/Attribute-based** controls for highly specific regulatory or data-sensitivity requirements.

Customizing authentication sessions

Customizing Authentication Sessions

In the Red Hat build of Keycloak (RHBK), authentication sessions represent the transient state of a user during the login process. Managing these sessions effectively is critical for both security and user experience, ensuring that authentication flows are tracked reliably without leaving dangling processes or creating vulnerabilities.

Understanding Authentication Sessions

When a user initiates a login request, Keycloak creates an **Authentication Session**. This session tracks the state of the authentication process, including which flow is being executed, the progress through the flow, and any errors or warnings encountered.

Keycloak distinguishes between different types of sessions to maintain state:

- **Authentication Session:** Tracks the state of the user **while** they are authenticating (e.g., verifying credentials, multi-factor authentication steps).
- **User Session:** Created once authentication succeeds. This persists for the duration of the user's active login, allowing them to access multiple applications via Single Sign-On (SSO).
- **Client Session:** A sub-component of the user session that tracks the user's status within a specific client application.

Configuring Session Limits and Expiration

To prevent resource exhaustion and mitigate session hijacking risks, you can customize the lifecycle of these sessions.



Authentication sessions are distinct from active user sessions. While user sessions track the logged-in state, authentication sessions track the **login attempt**. Limiting these helps defend against brute-force attacks and stale session accumulation.

Key Configuration Parameters

You can manage session behavior through the Admin Console or the Admin CLI. Key settings include:

- **SSO Session Idle:** The time an SSO session can be idle before it expires.
- **SSO Session Max:** The maximum time an SSO session can exist, regardless of activity.
- **Client Session Idle:** The time a client-specific session can be idle.
- **Client Session Max:** The maximum time a client-specific session can exist.

Guided Exercise: Customizing Authentication Session Lifetimes

In this exercise, you will modify the session settings for a specific realm to balance security requirements with user convenience.

Log in to the Admin Console.

1. Navigate to **Realm settings** in the left-hand menu.

2. Select the **Sessions** tab.
3. Locate the **SSO Session Idle** field and set it to **30 minutes**.
4. Locate the **SSO Session Max** field and set it to **10 hours**.
5. Click **Save**.



Changing these values applies to all users within the realm. Ensure that your security policies align with the business requirements for session duration.

Troubleshooting Session Issues

If users report being logged out unexpectedly, consider the following:

1. **Cache Configuration:** Authentication sessions are stored in the database but managed via caches (**sessions** and **clientSessions**). If these caches are misconfigured or under-provisioned in a clustered environment, session data may be lost.
2. **Token Revocation:** If a client application performs a token revocation, Keycloak immediately destroys the associated client session. Verify if your application is triggering unnecessary revocation calls.
3. **Clock Skew:** Ensure all nodes in your Keycloak cluster have synchronized clocks. Significant clock drift can cause session validation to fail prematurely.

Lab: Authentication and Authorization

In the broader context of this module, you will now observe how these session configurations impact the user's flow.

1. Open a browser and initiate a login flow for a test client.
2. Observe the session in the **Sessions** tab of the Admin Console for that user.
3. Modify the **SSO Session Idle** time to a very low value (e.g., **1 minute**).
4. Authenticate and remain idle for longer than 60 seconds.
5. Attempt to access a protected resource. You should be prompted to re-authenticate, confirming that the session has expired per your custom configuration.

Guided Exercise: Authenticate Users with Red Hat build of Keycloak

Guided Exercise: Authenticate Users with Red Hat build of Keycloak

In this exercise, you will configure a basic authentication flow within the Red Hat build of Keycloak. Authentication is the process of verifying the identity of a user, and Red Hat build of Keycloak manages this through configurable authentication flows.

Prerequisites

- A running instance of Red Hat build of Keycloak (version 26.6).
- Access to the Admin Console with administrative privileges.
- A pre-configured realm (e.g., `demo-realm`).

Objectives

- Configure a user within the realm.
- Test the default browser-based login flow.
- Observe the interaction between the application and the Keycloak authentication server.

Step 1: Create a Test User

Before testing authentication, you must ensure at least one user exists in your target realm.

1. Log in to the Keycloak Admin Console.
2. Select your realm from the dropdown menu in the top-left corner.
3. In the left navigation menu, click **Users**.
4. Click **Add user**.
5. Enter a username (e.g., `test-user`), an email address, and ensure "Email verified" is set to **Off** for testing purposes. Click **Create**.
6. Select the **Credentials** tab for your new user.
7. Click **Set password**.
8. Enter a password, toggle **Temporary** to **Off** (to avoid a forced reset), and click **Save**. Confirm the change when prompted.

Step 2: Validate the Authentication Flow

Red Hat build of Keycloak uses "Authentication Flows" to determine how a user proves their identity.

1. In the Admin Console, click **Authentication** in the left navigation menu.
2. Review the list of flows. The **Browser** flow is the default flow used for standard web application logins.
3. Observe the executions within the Browser flow (e.g., `Cookie`, `Kerberos`, `Username Password Form`). This is the sequence of actions that occurs when a user attempts to log in.

Step 3: Test Authentication via the Account Console

The Account Console is an excellent way to verify that your authentication configuration is functioning correctly.

1. Open your browser and navigate to the Account Console URL: `http://<KEYCLOAK_HOST>:<PORT>/realms/<REALM_NAME>/account/`

2. You will be redirected to the Keycloak login page.
3. Enter the credentials for `test-user` created in Step 1.
4. If successful, you will be directed to the Account Management dashboard. This confirms that the identity provider successfully authenticated the user against the credentials stored in the database.

Troubleshooting Tips

- **Login Failure:** If the user is rejected, check the **Events** tab in the Admin Console to view error logs associated with the failed login.
- **Realm Mismatch:** Ensure you are testing against the specific realm where you created the user.
- **Browser Cache:** If you are testing changes to authentication flows, open an Incognito/Private window to ensure no previous session cookies interfere with the testing process.

Summary

You have successfully created a user and verified that the default authentication flow correctly authenticates the user against the Red Hat build of Keycloak. In the next section, you will learn how to secure specific resources by enforcing authorization policies for these authenticated users.

Guided Exercise: Authorize Users with Red Hat build of Keycloak

Guided Exercise: Authorize Users with Red Hat build of Keycloak

In this exercise, you will learn how to implement Role-Based Access Control (RBAC) to authorize users within the Red Hat build of Keycloak. You will define roles, assign them to users, and verify that these permissions are correctly reflected in the user's security tokens.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- A configured Realm and at least one test user account.
- Access to the Keycloak Admin Console.

Step 1: Create Realm Roles

Roles define the set of permissions a user possesses within your application or system.

1. Log in to the **Admin Console**.
2. Select your target **Realm** from the dropdown menu in the top-left corner.
3. In the left navigation menu, click **Realm roles**.
4. Click **Create role**.
5. Provide a name for the role (e.g., `app-user` or `manager`) and an optional description.

6. Click **Save**.

Step 2: Assign Roles to Users

Once roles are defined, you must map them to specific users to grant access.

1. Navigate to **Users** in the left navigation menu.
2. Click on the **Username** of the user you wish to authorize.
3. Navigate to the **Role mapping** tab.
4. Click **Assign role**.
5. Select the role you created in the previous step (e.g., **manager**).
6. Click **Assign**. The role is now associated with the user.

Step 3: Verifying Role Inclusion in Tokens

When a user authenticates, Keycloak embeds these roles into the Identity and Access tokens. To verify:

1. Use an OIDC debugger or a client application to initiate a login flow for the user.
2. Once the login is successful, decode the received **Access Token** (JWT).
3. Inspect the **realm_access** claim within the payload.
4. You should see your defined role listed under the **roles** array:

```
"realm_access": {
  "roles": [
    "manager",
    "offline_access",
    "uma_authorization"
  ]
}
```

Step 4: Configure Client-Level Authorization (Optional)

If your application requires specific permissions separate from global realm roles, use Client Roles:

1. Navigate to **Clients** in the left menu.
2. Select your target **Client**.
3. Navigate to the **Roles** tab within the client configuration.
4. Create a client-specific role.
5. Assign this role to a user via the **User > Role mapping** screen, ensuring the "Filter by clients" toggle is enabled.

Troubleshooting Tips

- **Token Refresh:** If you update a user's role while they have an active session, they must re-authenticate or refresh their token for the changes to take effect in their access token.
- **Role Scopes:** Ensure your Client Scopes include the **roles** mapper. By default, the **roles** protocol mapper is included in the **openid** scope, which automatically adds realm and client roles to the token.

Summary

You have successfully defined roles, mapped them to a user, and confirmed their presence in the security token. This setup allows your downstream applications to perform authorization checks based on the **realm_access** claims provided by Keycloak.

Guided Exercise: Customize authentication with Red Hat build of Keycloak

Guided Exercise: Customize authentication with Red Hat build of Keycloak

In this exercise, you will modify the default authentication behavior of your realm to enhance security. By default, Red Hat build of Keycloak provides a "Browser" flow that handles standard user login. We will customize this flow to introduce additional requirements, such as forcing a password reset or adding an execution step.

Prerequisites

- Red Hat build of Keycloak 26.6 installed and running.
- Access to the Admin Console with administrative privileges.
- A realm configured with at least one test user.

Task 1: Create a Custom Authentication Flow

Rather than modifying the built-in "Browser" flow directly (which is considered best practice to preserve defaults), you will duplicate it and create a custom version.

1. Log in to the **Admin Console**.
2. Select your target realm from the top-left dropdown.
3. In the left navigation menu, click **Authentication**.
4. Locate the **Browser** flow.
5. Click the **Action menu** (three dots) next to the Browser flow and select **Duplicate**.
6. Name the flow **Custom-Secure-Browser** and click **Save**.

Task 2: Adding an Authentication Execution

We will now add a "Terms and Conditions" requirement to the authentication flow to ensure users acknowledge security policies before accessing the application.

1. Within your **Custom-Secure-Browser** flow, locate the **Browser Forms** execution.
2. Click **Add step** in the Browser Forms section.
3. From the list, select **Terms and Conditions** and click **Add**.
4. Set the requirement for the new **Terms and Conditions** step to **Required**.
5. Ensure the step is positioned appropriately (usually placed after the username/password forms).

Task 3: Binding the Custom Flow

Now that the flow is defined, you must inform the realm to use this flow for browser-based logins.

1. Navigate back to the **Authentication** tab.
2. Click the **Action menu** next to the **Custom-Secure-Browser** flow.
3. Select **Bind flow**.
4. In the dialog, select **Browser Flow** from the dropdown and click **Save**.

Task 4: Testing the Customization

1. Open a new Incognito/Private browser window.
2. Navigate to your application or the Account Console login page.
3. Enter your test user credentials.
4. **Verification:** You should now be presented with a Terms and Conditions screen after entering your credentials, confirming that your custom authentication flow is active.

Troubleshooting

- **Flow not triggering:** Ensure the **Custom-Secure-Browser** flow is correctly bound to the "Browser Flow" binding in the Authentication tab.
- **Locked out of Admin Console:** If you make a mistake and lock your own login access, you can revert to the default flow using the **kcadm.sh** CLI tool or by temporarily creating a new realm with default settings to reset the binding.



To further enhance security, consider adding **OTP (One-Time Password)** or **WebAuthn** as a "Required" step in this flow to implement Multi-Factor Authentication (MFA) for all users in the realm.

Lab: Authentication and Authorization

Lab: Authentication and Authorization

In this lab, you will implement the core security mechanisms of Red Hat build of Keycloak. You will transition from basic user authentication to granular access control by defining roles and applying them to protected resources.

Objectives

- Configure a custom Authentication Flow.
- Define Roles and assign them to users.
- Implement Role-Based Access Control (RBAC) to authorize user access.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- An existing Realm created for your application.

Step 1: Configure Authentication Flows

Keycloak uses "Flows" to determine the sequence of steps a user must complete to be authenticated. In this step, you will modify the flow to enforce specific requirements.

1. Log in to the **Admin Console**.
2. Navigate to **Authentication** in the left-hand menu.
3. Click **Flows** and locate the **Browser** flow.
4. Click the **Actions** menu (three dots) next to the **Browser** flow and select **Duplicate**. Name it **Custom-Browser-Flow**.
5. Within **Custom-Browser-Flow**, change the **Username/Password Form** to **Required** and add an **OTP Form** as **Required** to enforce Multi-Factor Authentication (MFA).
6. Navigate to **Realm Settings** > **Authentication** tab and set the **Browser Flow** dropdown to **Custom-Browser-Flow**.

Step 2: Define Roles and Assign Users

Access control in Keycloak is primarily managed through roles.

1. Navigate to **Realm Roles** and click **Create role**.
2. Create two roles: **manager** and **employee**.
3. Navigate to **Users** and click **Add user**. Create a user (e.g., **jdoe**).
4. Under the **Role mapping** tab for **jdoe**, click **Assign role** and select **employee**.

Step 3: Implement Authorization

Now that roles are defined, you will restrict access to a sample application or protected endpoint.

1. Navigate to **Clients** and select your target application.
2. Enable **Authorization** in the client settings.
3. Navigate to the **Authorization** tab for the client:
 - **Resource**: Define a resource representing your API endpoint (e.g., **/api/reports**).
 - **Scope**: Define the allowed actions (e.g., **view**, **edit**).

- **Policy:** Create a **Role-Based Policy**. Select the **manager** role and associate it with the **edit** scope.
- **Permission:** Create a **Scope-based permission** that links your **manager** role to the **edit** scope of the **/api/reports** resource.

Step 4: Verification

To verify your configuration:

1. Attempt to log in as **jdoe** through the application.
2. Note that the system prompts for the OTP configured in Step 1.
3. Access the **/api/reports** endpoint with the **edit** scope.
4. **Expected Result:** The request should be denied (403 Forbidden) because **jdoe** only possesses the **employee** role, not the **manager** role required for editing.

Troubleshooting

- **Token Mismatch:** If authorization fails unexpectedly, ensure the **Audience** mapper is enabled in the Client Scope so the token includes the correct client ID.
- **Flow Not Triggering:** Ensure that the Realm Settings specifically point to the duplicated **Custom-Browser-Flow** rather than the default **Browser** flow.
- **Logs:** Check the Keycloak server logs for **WARN** messages related to **AUTH_FLOW** or **PERMISSION_DENIED** to identify which specific policy rejected the request.

Identity Brokering and User Federation

Identity Brokering and User Federation

Identity brokering and user federation in Red Hat build of Keycloak allow you to manage user identities across disparate systems effectively.

- **Identity Brokering:** Acts as an intermediary service that connects clients to external Identity Providers (IdPs). It delegates authentication and authorization to these providers, enabling users to access services using existing accounts from sources like social logins (e.g., Google, Facebook) or business partners.
- **User Federation:** Enables the integration of external user databases, such as LDAP or Active Directory, directly into Keycloak. This allows your applications to manage and authenticate users against existing enterprise directories without requiring manual migration or duplicate account creation.
- **Benefits:** These features improve the user experience by reducing login fatigue, enhance security by centralizing credential management, and eliminate the need for clients to store or handle sensitive user credentials directly.

LDAP and Active Directory integration

LDAP and Active Directory Integration

Red Hat build of Keycloak provides robust support for integrating existing enterprise identity stores, such as Lightweight Directory Access Protocol (LDAP) and Microsoft Active Directory (MSAD). By leveraging user federation, you can allow users to authenticate against their existing corporate credentials without migrating sensitive password data into the Keycloak database.

Understanding Identity Federation

Identity federation in Keycloak acts as a bridge. Keycloak queries the external LDAP server to validate credentials during the authentication process.

Key benefits include: * **Centralized Identity:** Maintain a single source of truth for user identities. * **Seamless SSO:** Users utilize their existing enterprise credentials to access applications secured by Keycloak. * **Synchronization:** User attributes can be synchronized into Keycloak, allowing for local role-based access control (RBAC) mapping.

Configuring LDAP Integration

To integrate an LDAP provider, you must configure the connection details, search filters, and user mapping attributes.

1. Navigate to your target realm in the Admin Console.
2. Click **User Federation** in the left sidebar.
3. Click **Add federation provider** and select **ldap**.

4. Configure the connection settings:

- **Edit Mode:** Choose **READ_ONLY** (recommended for strict directory control) or **WRITABLE** (if Keycloak should sync changes back to LDAP).
- **Connection URL:** The URI of your LDAP server (e.g., `ldap://ldap.example.com:389`).
- **Users DN:** The base DN where your users are located in the directory tree.
- **Bind DN/Password:** Credentials Keycloak uses to search the directory.

Integrating Microsoft Active Directory

When working with Microsoft Active Directory, specific mappers are often required to translate AD-specific attributes into standard OIDC/SAML claims. A common requirement is mapping the `userAccountControl` attribute, which handles user status flags like "disabled" or "locked."

Guided Exercise: Adding an MS Active Directory Mapper via Admin CLI

In this exercise, you will programmatically add an MS Active Directory user account control mapper using the `kcadm.sh` utility.



Ensure you have the `parentId` of your existing LDAP provider. You can retrieve this by running `kcadm.sh get components -r <realm-name>`.

1. Use the `kcadm.sh` tool to create the mapper component. Replace the `parentId` and `realm` values with your specific environment details.

```
$ kcadm.sh create components \  
-r demorealm \  
-s name=msad-user-account-control-mapper \  
-s providerId=msad-user-account-control-mapper \  
-s providerType=org.keycloak.storage.ldap.mappers.LDAPStorageMapper \  
-s parentId=b7c63d02-b62a-4fc1-977c-947d6a09e1ea
```

Explanation of the parameters: * `-s providerId=msad-user-account-control-mapper`: Specifies the specific mapper type for AD account control. * `-s providerType=org.keycloak.storage.ldap.mappers.LDAPStorageMapper`: Defines the Java class responsible for LDAP mapping operations. * `-s parentId`: Links this mapper directly to the LDAP provider instance created in the previous configuration step.

Troubleshooting Tips

- **Connectivity:** Use `ldapsearch` from the Keycloak host to verify that the LDAP/AD server is reachable and that your Bind DN credentials are correct.
- **Mapping Mismatch:** If users appear in Keycloak but cannot log in, verify that the "Username LDAP Attribute" matches the attribute being queried (e.g., `sAMAccountName` for AD or `uid` for OpenLDAP).
- **Logs:** Enable `DEBUG` level logging for the `org.keycloak.storage.ldap` category if synchronization fails.

Delegating authentication to third-party identity providers

Delegating Authentication to Third-Party Identity Providers

In modern distributed architectures, maintaining separate user credentials for every application creates friction for users and management overhead for administrators. Red Hat build of Keycloak addresses this through **Identity Brokering**, a mechanism that allows you to delegate authentication and authorization to external Identity Providers (IdPs).

Conceptual Overview

An Identity Broker acts as an intermediary service. Instead of forcing a user to register locally within your Keycloak realm, the broker redirects the user to a trusted external entity to verify their identity. Once the external IdP validates the user, it sends a token (or assertion) back to Keycloak, which then grants the user access to your internal services.

This approach offers several strategic benefits:

- * **Improved User Experience:** Users can authenticate using accounts they already possess (e.g., Google, GitHub, or a corporate partner's IdP).
- * **Credential Isolation:** Your local applications never handle the user's external credentials directly.
- * **Centralized Management:** You manage the trust relationship with the external IdP in one place, rather than configuring every individual application to talk to every external provider.

Supported Protocols

Red Hat build of Keycloak supports delegation through standardized protocols. When configuring a third-party provider, you must select the protocol that the external IdP supports:

- **SAML v2.0:** Commonly used in enterprise environments to facilitate Single Sign-On (SSO) between business partners.
- **OpenID Connect (OIDC) / OAuth 2.0:** The industry standard for modern web and mobile applications, utilized by virtually all major social providers (Google, Facebook) and cloud identity services.

Guided Exercise: Integrating an External Identity Provider

In this exercise, you will configure an external OIDC-based identity provider to delegate user authentication.

Prerequisites

- Access to a running Red Hat build of Keycloak instance.
- An account with an external OIDC provider (e.g., Google Cloud Console or a mock OIDC server).

Step-by-Step Configuration

1. **Access the Admin Console:** Log in to your Keycloak Admin Console and select the target **Realm**.

2. **Navigate to Identity Providers:** In the left-hand menu, click **Identity providers**.
3. **Add Provider:** Click the **Add provider** dropdown and select the desired protocol (e.g., **OpenID Connect**).
4. **Configure Provider Details:** Enter the required metadata provided by your external IdP:
 - **Alias:** A unique identifier for the provider (e.g., **google-idp**).
 - **Client ID:** The ID issued by the external provider.
 - **Client Secret:** The secret key associated with your Client ID.
 - **Discovery Endpoint:** The URL provided by the external IdP to automatically fetch configuration endpoints.
5. **Save and Verify:** Click **Save**. Keycloak will generate a **Redirect URI**. Copy this URI and paste it into your external IdP's configuration (e.g., Google's "Authorized redirect URIs" field).

Lab: Authenticating via a Third-Party IdP



Ensure that the "First Login Flow" is configured correctly in the Identity Provider settings. This flow determines how Keycloak handles users who are authenticating for the first time via the external provider (e.g., automatically creating a local user account or forcing a manual review).

Activity: 1. Open your application URL in an incognito/private browser window. 2. Observe the login page; you should see the name of your configured Identity Provider (e.g., "Sign in with Google"). 3. Click the provider button and follow the external login prompts. 4. Once authenticated by the third-party, verify that you are successfully redirected back to the application and automatically logged in. 5. Check the **Users** section in the Admin Console to confirm that the user has been successfully provisioned within your realm.

Social login configuration

Social Login Configuration

Social login, also known as social authentication, allows your users to authenticate into your applications using credentials from established third-party identity providers (IdPs). By leveraging Red Hat build of Keycloak (RHBK) as an identity broker, you eliminate the need for users to manage yet another set of credentials, while simultaneously reducing the burden of credential storage and management for your organization.

Overview of Social Identity Providers

RHBK provides out-of-the-box support for numerous social networks. When you configure these providers, RHBK acts as the intermediary, delegating the authentication process to the trusted provider.

Supported providers include: * **Social Networks:** Facebook, Twitter, LinkedIn, Instagram * **Developer Platforms:** GitHub, GitLab, Bitbucket, Stack Overflow * **Cloud & Enterprise:** Google, Microsoft, OpenShift v4, PayPal

Configuration Parameters

When adding an identity provider, several key settings determine how the authentication flow and data synchronization behave. Understanding these parameters is crucial for a seamless integration:

- **GUI Order:** A numerical value that defines the order in which the provider appears on the login page. Lower numbers appear first.
- **First Login Flow:** Defines the authentication flow executed after the very first successful login. This is typically used to capture additional information or verify user identity.
- **Post Login Flow:** Defines the flow executed after **every** successful login. This is useful for enforcing multi-factor authentication (MFA) or session-specific checks.
- **Sync Mode:** Determines how user attributes are synchronized between the IdP and RHBK:
- **legacy:** Maintains standard RHBK behavior regarding mappers.
- **import:** Imports user data from the provider only during the initial account creation.
- **force:** Updates user data from the provider every single time the user logs in.

Procedure: Configuring a Social Identity Provider

To enable social login for your realm, follow these steps. Note that while the steps below use Facebook as an example, the workflow remains consistent across all supported providers, though specific technical requirements (such as Client IDs and Secrets) vary per provider.

1. Navigate to the **Admin Console**.
2. In the left-hand menu, click **Identity Providers**.
3. Click the **Add provider** dropdown list.
4. Select the desired provider (e.g., **Facebook**).
5. On the configuration page, provide the required credentials (e.g., Client ID and Client Secret) obtained from your provider's developer portal.
6. Configure the **Sync Mode** and **Flow** settings as required by your security policy.
7. Click **Save**.



If you are unsure what a specific setting does, hover over the question mark (?) symbol located next to each configuration option. This provides in-app context and descriptions for the field.

Guided Exercise: Integrating Social Login

Objective

Configure GitHub as an identity provider to allow users to authenticate into your RHBK realm using their GitHub accounts.

Steps

1. **Prepare the Provider:** Log in to your GitHub account, navigate to **Settings > Developer**

Settings > OAuth Apps, and create a new application. Note the **Client ID** and the **Client Secret** generated.

2. Add the Provider in RHBK:

- Go to **Identity Providers** in your RHBK Admin Console.
- Select **GitHub** from the **Add provider** list.
- Enter the **Client ID** and **Client Secret** into the respective fields.

3. Set Sync Mode: Set the **Sync Mode** to **import** to ensure user profile information is pulled into the local RHBK user database upon first login.

4. Verification:

- Open an incognito browser window and navigate to your application's login page.
- Verify that a "Sign in with GitHub" button (or link) appears on the login screen.
- Attempt to log in to ensure the flow redirects correctly and authenticates the user.

Guided Exercise: Federating Users with Red Hat build of Keycloak

Guided Exercise: Federating Users with Red Hat build of Keycloak

In this exercise, you will connect Red Hat build of Keycloak to an external User Federation provider. This allows you to manage users in an existing LDAP directory while leveraging Keycloak's advanced authentication features.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- Access to the Admin Console.
- An accessible LDAP server (e.g., OpenLDAP or Active Directory).

Task 1: Create an LDAP User Federation Provider

In this task, you will configure Keycloak to communicate with your LDAP directory.

1. Log in to the Keycloak Admin Console.
2. Select your target realm from the top-left dropdown menu.
3. In the left navigation menu, click **User Federation**.
4. Click the **Add provider** dropdown and select **ldap**.
5. Configure the following fields based on your LDAP infrastructure:
 - **Console Display Name:** **My-Corporate-LDAP**
 - **Import Users:** **ON** (This copies LDAP users into the local Keycloak database).
 - **Connection URL:** **ldap://<ldap-server-hostname>:<port>**

- **Users DN:** `ou=users,dc=example,dc=com` (The base DN for your users).
- **Bind DN:** `cn=admin,dc=example,dc=com` (The DN used to bind to the LDAP server).
- **Bind Credential:** `**` (The password for the Bind DN).

6. Click **Save**.

Task 2: Verify Connection and Test Synchronization

Before finalizing, verify that Keycloak can successfully communicate with the LDAP server.

1. Within the **My-Corporate-LDAP** provider configuration, click the **Connection** tab.
2. Click **Test connection**. If successful, a confirmation message appears.
3. Click **Test authentication** to ensure the Bind credentials are correct.
4. Navigate to the **Mappers** tab.
5. Click **Add mapper** and select **By template**.
6. Select common mappers like `username`, `email`, and `first name` to ensure user attributes map correctly to Keycloak attributes.

Task 3: Import and Verify Users

Now that the connection is established, you will verify the visibility of federated users.

1. In the left navigation menu, click **Users**.
2. Click **View all users**.
3. Look for users originating from the LDAP source.
 - **Note:** If you selected **Import Users**, these users are now searchable in the Keycloak database.
4. Select an imported user. Note the **IDP Link** field, which indicates the user is federated via the LDAP provider.

Troubleshooting Tips

- **Connectivity Issues:** Ensure your Keycloak server has network reachability to the LDAP port (typically 389 or 636 for LDAPS).
- **Permission Denied:** Verify that the **Bind DN** has sufficient read permissions on the **Users DN** subtree.
- **Sync Errors:** If users are not appearing, click the **Settings** tab in your LDAP provider and click **Synchronize all users** to trigger a manual import.

Summary

You have successfully configured an identity provider to federate users into Red Hat build of Keycloak. This setup allows you to maintain a single source of truth for user identities while providing centralized access control via Keycloak.

Guided Exercise: Integrating Social Login with Red Hat build of Keycloak

Guided Exercise: Integrating Social Login with Red Hat build of Keycloak

In modern web application development, users often prefer the convenience of using existing social media accounts to access services rather than creating new credentials. Red Hat build of Keycloak simplifies this by acting as an identity broker, allowing you to delegate authentication to third-party providers like Google, GitHub, or Facebook.

In this exercise, you will configure an external identity provider to allow users to authenticate to your Keycloak-protected applications using their existing social credentials.

Prerequisites

- A running instance of Red Hat build of Keycloak (Development mode).
- Access to the Keycloak Admin Console.
- A registered developer account with a third-party provider (e.g., [Google Cloud Console](<https://console.cloud.google.com/>)).

Step 1: Create an OAuth 2.0 Client at the Identity Provider

Before Keycloak can communicate with a social provider, you must register your application with that provider to obtain credentials.

1. Log in to your chosen Identity Provider's developer portal (e.g., Google Cloud Console).
2. Create a new **OAuth 2.0 Client ID**.
3. Set the **Authorized Redirect URI** to the following format: https://<KEYCLOAK_HOST>/realms/<REALM_NAME>/broker/google/endpoint
4. Copy the generated **Client ID** and **Client Secret**. You will need these for the Keycloak configuration.

Step 2: Configure the Identity Provider in Keycloak

Now, you will configure Keycloak to recognize the provider.

1. Log in to the Keycloak Admin Console.
2. From the **Realm** dropdown, select the realm you are using for your application.
3. In the left-hand menu, click **Identity providers**.
4. Click the **Add provider** dropdown and select your provider (e.g., **Google**).
5. In the **Add identity provider** screen:
 - **Client ID:** Paste the ID obtained in Step 1.
 - **Client Secret:** Paste the secret obtained in Step 1.
6. Click **Add**.

Step 3: Verify the Integration

Once saved, you must ensure the integration is active and accessible to end-users.

1. Navigate to the **Mappers** tab within your new Identity Provider configuration.
2. By default, Keycloak maps essential user information (like email and username) from the provider to your local realm database. You can add additional mappers if you need to capture specific claims (e.g., profile picture or custom attributes).
3. Open an Incognito window and navigate to your application's login page.
4. You should now see the configured social provider button displayed under the standard login form.

Troubleshooting Tips

- **Redirect URI Mismatch:** If the login fails immediately, ensure that the redirect URI configured in the Identity Provider matches the Keycloak endpoint exactly, including the protocol (HTTPS vs HTTP).
- **Clock Skew:** Ensure your server time is synchronized. Identity providers may reject tokens if the time difference between the server and the provider is significant.
- **Consent Screens:** Ensure that in your developer console, the OAuth consent screen is configured as "Published" or "Testing" with your user email added as a test user.

Summary

You have successfully configured Red Hat build of Keycloak as an identity broker. Your users can now leverage social login, reducing the friction of onboarding and centralizing authentication management within your Keycloak environment.

Lab: Identity Brokering and User Federation

Lab: Identity Brokering and User Federation

In this lab, you will configure Red Hat build of Keycloak to act as a central identity hub. You will implement external identity integration by configuring user federation to a directory service and enabling social login providers. This architecture allows you to consolidate authentication sources and streamline access management for your applications.

Objectives

- Configure an LDAP/Active Directory user federation provider.
- Configure a social login identity provider (e.g., GitHub or Google).
- Verify user authentication through brokered and federated sources.

Prerequisites

- A running instance of Red Hat build of Keycloak.

- Access to an LDAP/Active Directory server (or a mock service).
- Credentials for an external Identity Provider (for social login).

Lab Activities

Task 1: Configure User Federation

In this task, you will connect Keycloak to an LDAP directory to allow existing corporate users to authenticate without manual migration.

1. Log in to the **Keycloak Admin Console**.
2. Select the **Realm** you want to configure.
3. In the left navigation menu, click **User Federation**.
4. Click **Add provider** and select **ldap**.
5. Configure the following mandatory fields:
 - **Edit mode**: Select **READ_ONLY** (or **WRITABLE** if LDAP write-back is required).
 - **LDAP URL**: Enter the connection string (e.g., **ldap://ldap.example.com:389**).
 - **Users DN**: Define the base DN where users are stored.
 - **Bind Type**: Set to **simple**.
6. Click **Save**.
7. Click the **Test connection** button to ensure Keycloak can communicate with the directory.
8. Navigate to the **Users** tab and verify that federated users are visible (or searchable via the "View all users" button).

Task 2: Configure Identity Brokering (Social Login)

In this task, you will enable social login to delegate authentication to a trusted third party.

1. In the **Keycloak Admin Console**, navigate to **Identity providers**.
2. In the **Add provider** dropdown, select your desired provider (e.g., **GitHub**).
3. Enter the **Client ID** and **Client Secret** obtained from the external provider's developer portal.
4. Ensure the **Redirect URI** matches the one provided in the configuration screen; copy this into your social provider's application settings.
5. Click **Add**.
6. Navigate to your application and attempt a login. You should now see the social provider button on the login screen.

Verification

To verify your configuration:

1. **Test LDAP**: Attempt to log in to the account console using a user account that exists only in your LDAP directory.

2. **Test Social Login:** Click the social provider button on the login page, authenticate via the external service, and confirm that the user is successfully redirected back to the Keycloak application with a valid session.
3. **Audit:** In the Admin Console, go to **Users** and verify that the federated/brokered users have been automatically created in the Keycloak local database cache (or linked accordingly).

Troubleshooting

- **LDAP Connection Refused:** Verify that the Keycloak server has network reachability to the LDAP server and that firewalls are not blocking port 389 (LDAP) or 636 (LDAPS).
- **Social Login Redirect Errors:** Ensure the **Redirect URI** in the identity provider configuration exactly matches the URI registered in the external provider's developer console (including trailing slashes).
- **Mapper Issues:** If user attributes (like email or name) are not appearing, review the **Mappers** tab for your identity provider/user federation to ensure the attribute names match your external schema.

Additional Resources

- [Red Hat build of Keycloak Documentation](#)
- [Identity Brokering and User Federation Overview](#)

Red Hat build of Keycloak on Red Hat OpenShift

Red Hat build of Keycloak on Red Hat OpenShift

Deploying the Red Hat build of Keycloak on Red Hat OpenShift leverages containerization to provide a scalable, highly available identity and access management solution. This integration ensures seamless authentication and authorization for applications running within the OpenShift ecosystem.

Key aspects include:

- **Containerized Deployment:** Running Keycloak as a containerized workload to ensure consistent environments and simplified lifecycle management.
- **OpenShift Integration:** Utilizing OpenShift's native capabilities, such as routes, services, and secrets, for secure networking and configuration management.
- **Scalability and Resilience:** Taking advantage of OpenShift's orchestration to handle identity traffic efficiently and maintain service availability.
- **Configuration Management:** Implementing custom configurations and security policies tailored for containerized environments.

Deploying Keycloak in containerized environments

Deploying Red Hat build of Keycloak in Containerized Environments

In modern infrastructure, the Red Hat build of Keycloak is designed to be cloud-native and immutable. Deploying in a containerized environment—such as Podman or Kubernetes—requires a shift from traditional installation methods to a focus on container image optimization and environment variable configuration.

Containerization Concepts for Keycloak

Running Red Hat build of Keycloak as a container ensures consistency across development, testing, and production environments. Key operational considerations include:

- **Immutable Infrastructure:** Containers should be treated as ephemeral. Rather than patching a running container, you should re-provision a new container from an updated image.
- **Startup Performance:** To ensure containers start rapidly, it is recommended to build an optimized image. This involves pre-configuring the server, including setting up build-time features and optimizing the Quarkus engine, rather than performing these tasks at runtime.
- **Networking:** Containers typically isolate the application. You must explicitly map ports (e.g., `8080`) to the host network to allow external traffic to reach the Keycloak service.

Running a Standard Container

If your environment does not require highly optimized startup times, you can run a standard Red Hat build of Keycloak image.

```
podman run --name mykeycloak -p 127.0.0.1:8080:8080 \
  quay.io/keycloak/keycloak:latest \
  start-dev
```



The `start-dev` mode is intended for development purposes only. For production, you must use the `start` command and provide the necessary production configuration, such as database connections and TLS certificates.

Providing Initial Admin Credentials

By default, the Red Hat build of Keycloak restricts the creation of the initial administrator user to local network connections. In a containerized environment, where the loopback interface is restricted to the container namespace, you must provide the initial admin credentials via environment variables during the first startup.

If you fail to provide these, the Admin Console will remain inaccessible.

Guided Exercise: Launching a Keycloak Container with Admin Access

In this exercise, you will deploy a containerized instance of Red Hat build of Keycloak with a pre-configured administrator account.

1. Prerequisites

- A container engine (Podman or Docker) installed.
- Access to the `quay.io/keycloak/keycloak` registry.

2. **Step 1: Run the container with environment variables** Execute the following command to define the administrator username and password:

```
podman run --name mykeycloak -p 8080:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin \
  -e KC_BOOTSTRAP_ADMIN_PASSWORD=password \
  quay.io/keycloak/keycloak:latest \
  start-dev
```

1. Step 2: Verify the installation

- Open your browser and navigate to <http://localhost:8080>.
- Click on **Administration Console**.
- Log in using the credentials defined in the environment variables above (`admin` / `password`).

Summary

When deploying Keycloak in containers: * **Always use environment variables** to define the initial admin account to bypass local-only restriction constraints. * **Prioritize image optimization** for production environments to ensure fast re-provisioning and reduced resource overhead. * **Map container ports** correctly to ensure the Admin Console and authentication endpoints are reachable.

Configuration and management on OpenShift

Configuration and management on OpenShift

Once the Red Hat build of Keycloak is deployed on Red Hat OpenShift, ongoing configuration and management are essential to ensure the stability, security, and scalability of your identity and access management (IAM) infrastructure.

Operational Management on OpenShift

Managing Red Hat build of Keycloak in a containerized environment shifts the focus from traditional server maintenance to Kubernetes-native operations. Key configuration aspects include:

- **Custom Resource Definitions (CRDs):** Red Hat build of Keycloak on OpenShift is typically managed via the Keycloak Operator. You manage the lifecycle of your Keycloak instances by updating the **Keycloak** Custom Resource (CR).
- **Secret Management:** Sensitive data, such as database credentials and TLS certificates, must be stored in OpenShift **Secrets** and referenced within your Keycloak deployment manifests.
- **Horizontal Scaling:** Use the OpenShift Horizontal Pod Autoscaler (HPA) to scale the number of Keycloak pods based on CPU or memory utilization.
- **Persistence:** Ensure that your underlying database (such as PostgreSQL) uses **PersistentVolumeClaims** (PVCs) to maintain state across pod restarts or redeployments.

Guided Exercise: Configuring Keycloak on OpenShift

This exercise guides you through verifying your deployment and updating configuration parameters using the OpenShift CLI.

Prerequisites

- Ensure you are logged into your cluster via the CLI:

```
oc login https://api.ocp4.example.com:6443 -u admin -p redhat
```

Step 1: Verify the deployment status

Check that the Keycloak operator and your Keycloak instances are running correctly:

```
oc get pods -n rhssso
```

Step 2: Update configuration via the Keycloak CR

To modify Keycloak settings (e.g., changing the number of instances or environment variables), edit the Keycloak Custom Resource:

```
oc edit keycloak keycloak-instance -n rhssso
```

Step 3: Managing Secrets

If you need to rotate database credentials, update the corresponding Secret in the **rhssso** project:

```
oc create secret generic keycloak-db-secret \
  --from-literal=username=keycloak \
  --from-literal=password=newpassword \
  --dry-run=client -o yaml | oc apply -f -
```

Troubleshooting and Monitoring

Effective management requires visibility into the Keycloak logs and metrics.

- **Log Access:** To troubleshoot authentication errors or deployment failures, inspect the pod logs:

```
oc logs -l app=keycloak -n rhssso
```

- **Route Configuration:** Ensure that the OpenShift **Route** associated with your Keycloak instance is properly configured for TLS termination to ensure secure communication between your applications and the Keycloak server.



Always verify your changes in a staging environment before applying them to production. Use **oc diff** to compare your local configuration files with the cluster state to prevent accidental configuration drift.

Lab: Red Hat build of Keycloak on OpenShift

In this lab, you will perform a maintenance task on your running Keycloak instance.

1. **Task:** Scale your Keycloak deployment to two replicas.
2. **Action:** Modify the **instances** field in your **Keycloak** Custom Resource.
3. **Verification:** Confirm that two pods are running in the **rhssso** namespace using **oc get pods**.
4. **Validation:** Access the Admin Console via the route URL and verify that the Keycloak instance reflects the updated configuration.

Guided Exercise: Install Red Hat build of Keycloak on Red Hat OpenShift

Guided Exercise: Install Red Hat build of Keycloak on Red Hat OpenShift

In this exercise, you will deploy the Red Hat build of Keycloak on an OpenShift cluster using the Keycloak Operator. This approach leverages Kubernetes-native patterns to manage the lifecycle of your identity provider.

Prerequisites

- An active Red Hat OpenShift cluster.
- **oc** CLI tool installed and authenticated to your cluster.
- Access to the OperatorHub within the OpenShift web console.

Step 1: Create a Namespace

It is best practice to isolate your Keycloak deployment in its own project.

```
oc new-project sso-production
```

Step 2: Install the Keycloak Operator

The Keycloak Operator automates the deployment and management of Keycloak instances.

1. Log in to the OpenShift Web Console.
2. Navigate to **Operators > OperatorHub**.
3. Search for **Keycloak**.
4. Select the **Red Hat build of Keycloak** operator.
5. Click **Install**.
6. Ensure the **Installation Mode** is set to "A specific namespace on the cluster" and select **sso-production**.
7. Click **Install**.

Step 3: Define the Keycloak Custom Resource

Once the Operator is running, you must define the **Keycloak** custom resource. This resource specifies the desired state of your Keycloak deployment.

Create a file named **keycloak-instance.yaml**:

```
apiVersion: k8s.keycloak.org/v2alpha1
kind: Keycloak
metadata:
```

```
name: sso-keycloak
namespace: sso-production
spec:
  instances: 1
  image: registry.access.redhat.com/rhbk/keycloak-rhel9:22.0
  db:
    vendor: postgres
    host: keycloak-db
  hostname:
    hostname: sso.example.com
  tls:
    certificateSecret:
      secretName: keycloak-tls-secret
```

Apply the configuration:

```
oc apply -f keycloak-instance.yaml
```

Step 4: Verify the Installation

Check the status of your deployment to ensure the pods are starting correctly.

```
oc get pods -n sso-production
```

You should see the Keycloak pod transitioning to the **Running** state. You can inspect the logs to verify successful startup:

```
oc logs -f deployment/sso-keycloak -n sso-production
```

Troubleshooting Tips

- **Resource Constraints:** If the pod remains in **Pending** state, check the node resources using `oc describe pod <pod-name>`.
- **Image Pull Errors:** Ensure your cluster has appropriate access to the Red Hat Ecosystem Catalog or that your pull secrets are correctly configured.
- **Database Connectivity:** If the log shows connection failures, verify that your database service is running and that the **db** spec in your **Keycloak** custom resource matches the service name and credentials.

Summary

You have successfully installed the Red Hat build of Keycloak on OpenShift using the Operator pattern. You are now prepared to proceed to configuring the environment for authentication flows.

Guided Exercise: Configure Red Hat build of Keycloak on Red Hat OpenShift

Guided Exercise: Configure Red Hat build of Keycloak on Red Hat OpenShift

In this exercise, you will configure an existing Red Hat build of Keycloak deployment on OpenShift to ensure it is optimized for production readiness and secure communication.

Prerequisites

- A running Red Hat build of Keycloak instance deployed via the Keycloak Operator on OpenShift.
- Access to the OpenShift CLI (**oc**) with cluster-admin or project-admin privileges.

Task 1: Configuring TLS/SSL Certificates

By default, the Operator provides a self-signed certificate. For production environments, you must configure a custom certificate.

1. Create a secret containing your TLS certificate and key:

```
oc create secret tls keycloak-tls-secret \  
  --cert=path/to/tls.crt \  
  --key=path/to/tls.key -n keycloak-namespace
```

2. Update the **Keycloak** Custom Resource (CR) to reference the secret:

```
spec:  
  hostname:  
    hostname: sso.example.com  
  http:  
    tlsSecret: keycloak-tls-secret
```

Task 2: Environment Configuration via Keycloak CR

You can manage the configuration of your Keycloak instance by modifying the **Keycloak** Custom Resource. This ensures that your configuration is declarative and version-controlled.

1. Edit your Keycloak CR:

```
oc edit keycloak keycloak-instance -n keycloak-namespace
```

2. Add environment variables to tune performance, such as setting the cache stack:

```
spec:  
  instances: 3
```

```
additionalOptions:
  - name: cache
    value: ispn
  - name: cache-stack
    value: kubernetes
```

Task 3: Configuring the Database

While development mode uses the built-in H2 database, production environments require an external database like PostgreSQL.

1. Ensure the secret containing database credentials exists:

```
oc create secret generic keycloak-db-secret \
  --from-literal=username=keycloak \
  --from-literal=password=password123
```

2. Update the CR to point to your database service:

```
spec:
  db:
    vendor: postgres
    host: postgres-db-service
    usernameSecret:
      name: keycloak-db-secret
      key: username
    passwordSecret:
      name: keycloak-db-secret
      key: password
```

Task 4: Verifying the Configuration

After updating the CR, the Operator will automatically trigger a rolling update of the Keycloak pods.

1. Monitor the rollout:

```
oc rollout status statefulset/keycloak-instance -n keycloak-namespace
```

2. Confirm the new configuration is applied by checking the logs of the Keycloak pod:

```
oc logs -f keycloak-instance-0 -n keycloak-namespace
```



Always ensure that your **hostname** configuration matches the externally accessible route to prevent redirect mismatches during OIDC authentication flows.

Summary

You have successfully updated your Keycloak configuration on OpenShift by modifying the Custom Resource. This approach leverages the Operator pattern to manage the underlying stateful services, certificates, and database connections, ensuring a robust and reproducible environment.

Lab: Red Hat build of Keycloak on Red Hat OpenShift

Lab: Red Hat build of Keycloak on Red Hat OpenShift

This lab provides a hands-on environment to deploy and manage the Red Hat build of Keycloak within a Red Hat OpenShift cluster. You will utilize the Keycloak Operator to manage the lifecycle of your Keycloak instance.

Objectives

- Deploy the Keycloak Operator on Red Hat OpenShift.
- Provision a Keycloak instance using Custom Resources (CR).
- Expose the Keycloak service via an OpenShift Route.
- Verify the deployment and access the Admin Console.

Prerequisites

- Access to a Red Hat OpenShift cluster with `cluster-admin` or appropriate namespace permissions.
- The `oc` CLI tool authenticated to your cluster.
- Helm installed (optional, if using Helm charts for operator deployment).

Lab Activities

Step 1: Deploy the Keycloak Operator

The Keycloak Operator automates the installation, configuration, and management of Keycloak on OpenShift.

1. Create a new namespace for your Keycloak deployment:

```
oc new-project keycloak-system
```

2. Use the OpenShift Web Console or `oc` to install the **Keycloak Operator** from the OperatorHub.
3. Once installed, verify the operator status:

```
oc get pods -n keycloak-system
```

Step 2: Provision a Keycloak Instance

We define the Keycloak instance using a **Keycloak** Custom Resource. This instructs the operator to spin up the necessary pods, services, and configurations.

1. Create a YAML file named **keycloak-instance.yaml**:

```
apiVersion: k8s.keycloak.org/v2alpha1
kind: Keycloak
metadata:
  name: keycloak
  namespace: keycloak-system
spec:
  instances: 1
  image: quay.io/keycloak/keycloak:latest
  db:
    vendor: postgres
    host: keycloak-db
  hostname:
    hostname: keycloak.apps.cluster-domain.com
  tls:
    certificateSecret:
      secretName: keycloak-tls-secret
```

2. Apply the configuration:

```
oc apply -f keycloak-instance.yaml
```

Step 3: Expose and Verify

After the operator reconciles the state, verify that the Keycloak pod is running.

1. Check the deployment status:

```
oc get keycloak -n keycloak-system
```

2. Retrieve the initial admin credentials, which are stored in a generated Kubernetes secret:

```
oc get secret keycloak-initial-admin -n keycloak-system -o
jsonpath='{.data.username}' | base64 --decode
oc get secret keycloak-initial-admin -n keycloak-system -o
jsonpath='{.data.password}' | base64 --decode
```

3. Access the route created by the operator. You can find the URL by running:

```
oc get route keycloak -n keycloak-system
```

Troubleshooting Guidance

- **Pod CrashLoopBackOff:** Check the logs using `oc logs <pod-name> -n keycloak-system`. Often, this indicates a failure to connect to the database or incorrect environment variables.
- **Certificate Issues:** Ensure the `tls` secret referenced in the CR exists in the same namespace if you are providing custom certificates.
- **Operator Latency:** After applying the CR, it may take a few minutes for the Operator to pull the images and provision the resources. Use `oc get events` to track progress.

Review Questions

1. How does the Keycloak Operator simplify the update process for Keycloak compared to a manual installation?
2. Why is it recommended to use a persistent database (like PostgreSQL) instead of the embedded H2 database in an OpenShift production environment?
3. How can you modify the `Keycloak` Custom Resource to scale the number of instances for high availability?

Extending and Templating Keycloak

Extending and Templating Keycloak

Red Hat build of Keycloak provides flexibility through customization, allowing administrators to modify both the appearance and functionality of the platform.

- **Templating:** Customizes the user-facing interfaces, including login, registration, and account management pages, to align with organizational branding.
- **Extending:** Enhances the core functionality of the platform by implementing custom providers and features to meet specific business requirements.

Customizing user interfaces (Login, Registration, Account Management)

Extending and Templating Keycloak: Customizing User Interfaces

Red Hat build of Keycloak (RHBK) provides a powerful templating engine that allows organizations to align the user experience with their corporate branding and specific functional requirements. By customizing themes, you can modify the look and feel of login, registration, and account management screens without altering the core business logic of your applications.

Understanding Keycloak Themes

Keycloak uses a theme-based architecture to control the UI. A theme is a collection of templates (FreeMarker), styles (CSS), and resources (images, scripts) that define how users interact with the Identity and Access Management (IAM) system.

The primary theme types include:

- * **Login Theme:** Governs the login, OTP, registration, and forgotten password pages.
- * **Account Theme:** Governs the user's personal account management console.
- * **Email Theme:** Governs the automated emails sent by Keycloak (e.g., password reset, identity verification).

Dynamic Form Rendering and Validation

Modern Keycloak implementations leverage the User Profile SPI, which shifts UI customization from static template editing to dynamic schema-based rendering.

- **Declarative Attributes:** Instead of hardcoding fields into FreeMarker templates, you define user attributes in the Admin Console.
- **Dynamic Rendering:** RHBK automatically renders registration, update profile, and account management forms based on your defined attributes.
- **Built-in Validation:** You can apply validation rules (e.g., regex patterns, minimum length, required fields) per attribute. This ensures data integrity without requiring custom Java code for simple form logic.

Guided Exercise: Customizing the Login Theme

In this exercise, you will create a custom theme directory and override the default login page.

Prerequisites

- A running instance of Red Hat build of Keycloak.
- Access to the file system where Keycloak is deployed.

Step 1: Create the Theme Directory

1. Navigate to the `themes/` directory in your Keycloak installation.
2. Create a new directory for your custom theme: `mkdir -p mycompany/login`.
3. Create a `theme.properties` file inside `mycompany/login/`:

```
parent=keycloak
import=common/keycloak
```

Step 2: Override the Login Template

1. Copy the default login template to your custom directory:

`cp themes/keycloak/login/login.ftl themes/mycompany/login/login.ftl`
2. Edit `themes/mycompany/login/login.ftl` to add your company branding or custom HTML elements.
3. To apply the theme, log in to the **Admin Console**:
 - Navigate to **Realm Settings** > **Themes**.
 - Select your new theme (`mycompany`) under the **Login Theme** dropdown.
 - Click **Save**.

Best Practices for Templating

- **Use Parent Themes:** Always inherit from the `keycloak` theme using the `parent` property. This ensures your theme receives security patches and feature updates automatically.
- **Leverage CSS Variables:** Use CSS variables defined in the base theme to ensure consistent styling while minimizing the amount of custom CSS code you need to maintain.
- **Avoid Business Logic in Templates:** Use templates strictly for display. If you need complex logic, implement a custom `Authenticator` (via the Authentication SPI) rather than forcing logic into FreeMarker.
- **Dynamic Forms:** Prioritize the User Profile configuration in the Admin Console over modifying templates to ensure your forms remain maintainable as user requirements evolve.

Summary

Customizing the user interface in Red Hat build of Keycloak allows for seamless integration with existing organizational branding. By utilizing the dynamic User Profile system and the robust FreeMarker templating engine, you can create a secure, branded, and intuitive experience for your users while keeping the underlying authentication logic clean and maintainable.

Extending Keycloak functionality

Extending Red Hat build of Keycloak Functionality

Red Hat build of Keycloak is designed to be highly modular. While the platform provides comprehensive built-in features, organizations often have unique requirements—such as custom token validation, proprietary authentication logic, or unique data synchronization needs. Extending Keycloak allows you to integrate these custom behaviors seamlessly into the identity lifecycle.

Understanding Extensibility

You can extend Keycloak primarily by implementing its Service Provider Interface (SPI). An SPI is a contract that allows you to plug in custom code to replace or augment default Keycloak behaviors.

Key areas for extension include:

- * **Authentication:** Adding custom execution steps to authentication flows.
- * **User Storage:** Implementing custom providers to bridge Keycloak with legacy databases or external services.
- * **Protocol Mappers:** Transforming or adding custom claims to identity/access tokens.
- * **Adapters:** Creating custom client-side adapters for specialized environments.

Custom Adapters

In specific scenarios where the standard JavaScript adapter does not meet the requirements of your environment (e.g., non-browser environments or specialized frameworks), you can implement a custom adapter.

To create a custom adapter, you must implement the `KeycloakAdapter` interface. This ensures that your implementation satisfies all the lifecycle methods expected by the Keycloak client library.

Guided Exercise: Implementing a Custom Adapter

In this exercise, you will create a skeleton implementation of a custom adapter using TypeScript to ensure type safety.

Prerequisites

- `keycloak-js` library installed in your project.

Procedure

1. Create a new file named `custom-adapter.ts`.
2. Define your adapter by implementing the `KeycloakAdapter` interface:

```
import Keycloak, { KeycloakAdapter } from 'keycloak-js';

// Implement the 'KeycloakAdapter' interface
const MyCustomAdapter: KeycloakAdapter = {
  async login(options) {
    // Custom implementation for redirecting the user to the login page
    console.log("Custom login logic triggered", options);
  },
  async logout(options) {
    // Custom implementation for session termination
    console.log("Custom logout logic triggered", options);
  },
  async register(options) {
    // Custom implementation for registration flow
    console.log("Custom registration logic triggered", options);
  },
  async accountManagement() {
    // Custom implementation for account management
    console.log("Custom account management logic triggered");
  }
};
```

1. Initialize the Keycloak instance using your custom adapter:

```
const keycloak = new Keycloak({
  url: "http://keycloak-server",
  realm: "my-realm",
  clientId: "my-app"
});

// Pass the custom adapter into the init method
await keycloak.init({
  adapter: MyCustomAdapter,
});
```

Extending Authentication Flows

While Red Hat build of Keycloak provides robust built-in flows, these are immutable. To apply specific business logic, you utilize the **Authentication SPI**.

- **Custom Authenticators:** You can create custom authenticators that require additional credentials (e.g., custom challenge/response, SMS, or hardware tokens).
- **Execution Requirements:** Once deployed as a provider, these authenticators appear in the Admin Console. You can then add them to your Authentication Flows and set them to **REQUIRED**, **ALTERNATIVE**, or **DISABLED** to alter the flow's behavior without modifying the core system.

Best Practices for Extending Keycloak

- **Use the SPI:** Always prefer using the official SPIs over modifying internal code. This ensures your changes are compatible with future upgrades of the Red Hat build of Keycloak.
- **Performance Impact:** Custom providers run within the Keycloak server process. Ensure your custom code is optimized and does not introduce latency or memory leaks.
- **Logging:** Implement standard logging within your providers to ensure effective troubleshooting during authentication or integration failures.

Guided Exercise: Templating Keycloak

Guided Exercise: Templating Keycloak

In this exercise, you will learn how to customize the Red Hat build of Keycloak user interface by overriding the default FreeMarker templates. This allows you to align the authentication pages, registration forms, and account management consoles with your organization's specific branding requirements.

Objectives

- Understand the directory structure used for theme overrides.
- Create a custom theme directory in the Red Hat build of Keycloak server.
- Apply a custom logo and CSS to the login page.

Prerequisites

- A running instance of the Red Hat build of Keycloak.
- Access to the server's filesystem where Keycloak is installed.

Task 1: Create the Theme Directory Structure

Keycloak uses a theme-based system to control the look and feel. Themes are stored in the `themes/` directory within your Keycloak installation.

1. Navigate to the `themes/` directory in your Keycloak installation folder.
2. Create a new directory for your custom theme, for example, `mycompany`:

```
mkdir -p themes/mycompany/login/resources/css
mkdir -p themes/mycompany/login/resources/img
mkdir -p themes/mycompany/login/theme
```

Task 2: Configure the Theme Metadata

Keycloak requires a `theme.properties` file to identify the theme and define its parent theme. By setting a parent, you only need to override the files you want to change, inheriting the rest from the default base theme.

1. Create a file named `themes/mycompany/login/theme.properties` with the following content:

```
parent=base
import=common/keycloak
styles=css/styles.css
```

Task 3: Customize the Branding

Now, you will add a custom CSS file to override the default styles and add a logo.

1. Create `themes/mycompany/login/resources/css/styles.css`:

```
.login-pf body {
    background-color: #f0f0f0;
}

.login-pf .login-pf-page .card-pf {
    border-top: 5px solid #cc0000; /* Red Hat Red branding */
}
```

2. Place your company logo image in `themes/mycompany/login/resources/img/logo.png`.

Task 4: Apply the Theme

Once the files are in place, you must instruct Keycloak to use the new theme.

1. Log in to the **Admin Console**.
2. Select your **Realm** from the dropdown menu.
3. In the left-hand menu, navigate to **Realm settings**.
4. Click on the **Themes** tab.
5. In the **Login theme** dropdown, select `mycompany`.
6. Click **Save**.

Task 5: Verification

1. Open your browser and navigate to your Keycloak login page.
2. Observe that the background color has changed and the card styling now features the custom red border defined in your `styles.css`.
3. If the changes do not appear, ensure that the cache is disabled in your `conf/keycloak.conf` file during development:

```
# Set to true for development, false for production
http-relative-path=/
# Disable caching for easier theme development
```

```
cache=no
```

Note: After changing `keycloak.conf`, you must restart the server for the changes to take effect.

Summary

You have successfully created a custom theme, applied organization-specific branding through CSS, and configured the Realm to utilize this theme. This modular approach allows for maintainable UI updates without modifying the core Red Hat build of Keycloak source code.

Comprehensive Review

Comprehensive Review

The Comprehensive Review serves as the final assessment of the Red Hat build of Keycloak course. This module focuses on:

- **Practical Application:** A hands-on, gradable lab activity that evaluates knowledge gained across all course objectives.
- **Independent Problem Solving:** Participants must meet technical specifications without step-by-step guidance.
- **Skill Validation:** Reinforces key security concepts, installation procedures, and configuration management.
- **Guided Solutions:** Includes a comprehensive walk-through provided after the activity to ensure all course requirements are fully understood.

Review of key security concepts and configurations

Review of Key Security Concepts and Configurations

In this module, we consolidate the foundational security principles and configuration patterns required to harden and maintain a production-ready Red Hat build of Keycloak environment.

Security Concept Consolidation

Before deploying into high-trust environments, ensure you have mastered the following core security pillars:

- **Credential Isolation:** Understand how Keycloak acts as an Identity Provider (IdP) to centralize credentials, ensuring that individual applications never handle raw user passwords.
- **Token-Based Security:** Leverage OIDC and OAuth 2.0 to manage access. Always prioritize the use of short-lived **Access Tokens** and secure **Refresh Tokens** to maintain a balance between user convenience and security.
- **Least Privilege Authorization:** Utilize Keycloak's fine-grained Role-Based Access Control (RBAC) and Permission-Based Access Control to ensure users and services possess only the minimum required access rights.
- **Identity Federation & Brokering:** Centralize user management by delegating authentication to established LDAP/Active Directory stores or trusted third-party providers, reducing the attack surface by minimizing local user storage.

Critical Configuration Checklist

When configuring your Keycloak instances, verify these security-critical settings:

Configuration Area	Best Practice
Transport Layer	Always enforce TLS/HTTPS. Never expose the Admin Console or authentication endpoints over unencrypted connections.
Client Secrets	Store client secrets in secure vaults (e.g., HashiCorp Vault, OpenShift Secrets) rather than hardcoding them in configuration files.
Token Lifespans	Adjust Access Token Lifespan and SSO Session Idle timeouts to match your organization's risk profile. Shorter durations mitigate the impact of token interception.
Brute Force Detection	Enable and configure the built-in brute force protection settings to lock out accounts after repeated failed login attempts.
Audit Logging	Enable event logging to track authentication and administrative actions, facilitating compliance and forensic analysis.

Troubleshooting Guidance

If security mechanisms appear to be failing, use this diagnostic workflow:

1. **Inspect Token Claims:** Use the [JWT Debugger](#) to verify that the claims (roles, scopes, audience) within your generated tokens match your expected authorization policy.
2. **Examine Event Logs:** In the Admin Console, navigate to **Events > Login Events**. Filter for **LOGIN_ERROR** or **CODE_TO_TOKEN_ERROR** to identify patterns in authentication failures.
3. **Validate Redirect URIs:** A common source of OIDC failure is a mismatch in Redirect URIs. Ensure that the **Valid Redirect URIs** field in your Client configuration exactly matches the application's callback URL.

Knowledge Check

- **Scenario:** You notice unauthorized attempts to guess user passwords. Which Keycloak feature should be audited and tuned?
- **Scenario:** You need to integrate an external Active Directory server to prevent credential fragmentation. Which Keycloak subsystem handles this?
- **Review:** Why is "Credential Isolation" considered the primary security benefit of implementing an Identity Provider?



For a comprehensive review, return to the **Admin Console** and navigate to the **Realm Settings > Security Defenses** tab. Ensure your **Content-Security-Policy** and **Strict-Transport-Security** headers are configured according to your organization's security policy.

Lab: Install and Configure Red Hat build of Keycloak

Lab: Install and Configure Red Hat build of Keycloak

In this lab, you will perform a development-mode installation of the Red Hat build of Keycloak. This environment allows you to explore the administration console and verify basic server functionality.



This installation method is strictly for development and testing purposes. Development mode provides insecure defaults and should never be used in a production environment.

Prerequisites

- A system with **podman** installed and configured.
- Access to the Red Hat Ecosystem Catalog (registry.redhat.io).
- Sufficient permissions to pull container images and map local network ports.

Task 1: Installing the Server

You will use the **start-dev** command to launch the Keycloak container. This command configures the initial administrator credentials via environment variables.

1. Open your terminal.
2. Execute the following command to pull the image and start the container:

```
podman run --name mykeycloak \  
-p 127.0.0.1:8080:8080 \  
-e KC_BOOTSTRAP_ADMIN_USERNAME=admin \  
-e KC_BOOTSTRAP_ADMIN_PASSWORD=change_me \  
registry.redhat.io/rhbk/keycloak-rhel9:26.6 \  
start-dev
```

1. Wait for the server to initialize. You should see logs indicating the server is listening on **0.0.0.0:8080**.

Task 2: Accessing the Admin Console

Once the container is running, the Administration Console is available to manage your identity realm.

1. Open your preferred web browser.
2. Navigate to <http://localhost:8080>.
3. Click on the **Administration Console** link.
4. Log in using the credentials defined in the previous step:
 - **Username:** **admin**
 - **Password:** **change_me**

Task 3: Verifying the Configuration

After logging in, perform a brief verification of the environment:

1. Locate the **Master** realm dropdown in the top-left corner.
2. Navigate to **Realm settings** to observe the default configuration parameters.
3. Check the **Users** section to ensure you can view and add administrative user accounts.

Task 4: Cleaning Up

When you have finished the lab, stop and remove the container to free up system resources:

```
podman stop mykeycloak  
podman rm mykeycloak
```

Troubleshooting

- **Port Conflict:** If port **8080** is already in use, modify the **-p** parameter in the command (e.g., **-p 127.0.0.1:8081:8080**) to use a different host port.
- **Image Pull Error:** Ensure you are authenticated to the Red Hat registry by running **podman login registry.redhat.io** before attempting to pull the image.

Appendix

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)